OS Report 2026 — รหัสนิสิต 6710503755 เกียรติภูมิ พละสาร

บทนำ

รายงานนี้มี 2 ส่วน ส่วนที่ 1 คือโปรแกรมคูณเมตริกซ์แบบขนานด้วย C + OpenMP พร้อมการวัดประสิทธิภาพหลายมิติและการวิเคราะห์ด้วย Amdahl's Law ส่วนที่ 2 คือเครื่องมือ `mmap` (Rust) ที่แสดงภาพการใช้หน่วยความจำของโปรเซสจาก `/proc` ทั้งแบบเวลาจริงและแบบ snapshot การทดลองทั้งหมดทำบนเครื่องเดียวกัน

รายงานนี้รวบรวมกราฟทั้ง 12 รูปจาก `bench_results/figures` ของการทดสอบการคูณเมทริกซ์แบบขนานด้วย OpenMP (Task 1) โดยวัดเวลา, speedup, efficiency, GFLOP/s และหน่วยความจำ ภายใต้การตั้งค่าระดับ OS และคอมไพเลอร์ต่าง ๆ ตัวเลขทั้งหมดมาจาก `summary.md` และไฟล์ใน `raw/`

การทดลองทั้งหมดทำบนเครื่องเดียวกัน เมื่อวันที่ 12 ต.ค. 2026 (ข้อมูลจาก `sysinfo.txt` ที่ `bench.sh` บันทึกอัตโนมัติ)

รายการ	ค่า
CPU	AMD Ryzen 7 5800H — 8 คอร์จริง / 16 เธรด (SMT เปิด)
Cache	L1d 32 KiB/คอร์, L2 512 KiB/คอร์, L3 16 MiB (ใช้ร่วมกัน)
หน่วยความจำ	RAM 22 GiB, swap 8 GiB, NUMA node เดียว
NUMA	1 โหนด
OS	Linux 7.0.0-34-generic (Ubuntu 24.04)
คอมไพเลอร์	gcc 13.3.0 (libgomp), rustc/cargo 1.75.0
CPU governor	powersave, energy_perf_preference = balance_performance, boost เปิด
Transparent hugepages	madvise
Stack limit (`ulimit -s`)	8192 KiB
วันที่ทดสอบ	12 ต.ค. 2026
การตั้งค่า OS ตอนทดลอง	governor `powersave`, boost เปิด, SMT เปิด, THP = `madvise`, `ulimit -s` = 8192 KiB

CPU 0 และ CPU 1 เป็น SMT sibling ของ core เดียวกัน (`thread_siblings_list` = 0-1) ข้อเท็จจริงนี้อธิบายผลหลายอย่างในหัวข้อ 1.5

อัลกอริทึมที่เปรียบเทียบ: `naive` (ลูป ijk), `ikj` (สลับลำดับลูปให้เข้าถึงหน่วยความจำต่อเนื่อง) และ `blocked` (แบ่งบล็อกเพื่อใช้แคช) Speedup คิดเทียบกับอัลกอริทึมเดียวกันที่ 1 เธรด

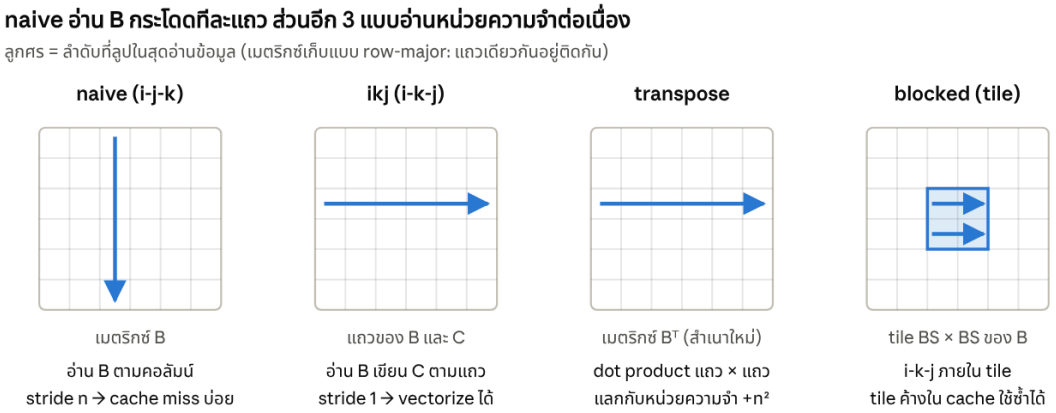
ส่วนที่ 1 — Intro to Parallel Programming

1.1 อัลกอริทึมที่ใช้

ทุกอัลกอริทึมคำนวณ C = A × B ของเมตริกซ์ n × n ชนิด `double` ด้วยจำนวนงานเท่ากันคือ $2n^3$ FLOP ต่างกันเฉพาะลำดับการเข้าถึงหน่วยความจำ ซึ่งเป็นตัวกำหนดความเร็วหลักของงานนี้

$$c_{ij} = \sum_{k=0}^{n-1} a_{ik}\, b_{kj}, \qquad \text{FLOP} = 2n^3, \qquad \text{GFLOP/s} = \frac{2n^3}{t_{mul} \cdot 10^9}$$

ภาษา C เก็บเมตริกซ์แบบ row-major สมาชิกในแถวเดียวกันจึงอยู่ติดกัน การอ่านตามแถวใช้ cache line ขนาด 64 ไบต์ (8 ค่า double) ได้ครบ ส่วนการอ่านตามคอลัมน์กระโดดทีละ 8n ไบต์ แต่ละ cache line ที่โหลดมาได้ใช้เพียง 1 ใน 8



รูป แบบการอ่านหน่วยความจำของลูปในสุด · 4 อัลกอริทึม

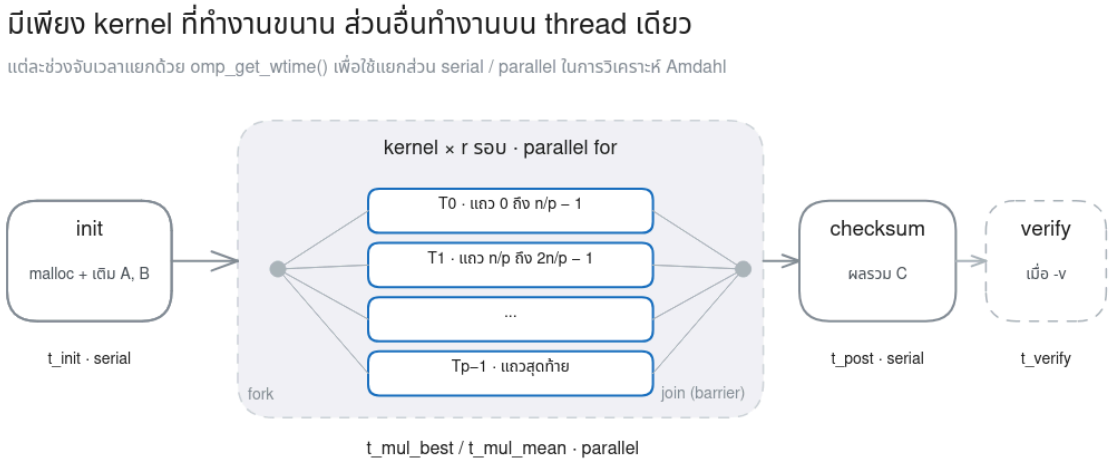
อัลกอริทึม (`-a`)	วิธีการ	หน่วยความจำ	เหตุผลที่เลือกทดลอง
`naive`	ลูป i-j-k ตามนิยาม	$3n^2 \times 8$ B	baseline สำหรับเทียบ
`ikj`	สลับลูปเป็น i-k-j (loop interchange)	$3n^2 \times 8$ B	เปลี่ยนลำดับลูปอย่างเดียวก็ได้ลูปในแบบ stride 1

อัลกอริทึม (-a)	วิธีการ	หน่วยความจำ	เหตุผลที่เลือกทดลอง
`transpose`	สร้าง B^T ก่อน (ทำแบบขนาน) แล้วทำ dot product ของแถว A กับแถว B^T	$4n^2 \times 8$ B	ดูการแลกหน่วยความจำเพิ่ม n^2 กับการอ่านแบบต่อเนื่อง
`blocked`	i-k-j ทีละ tile ขนาด BS × BS (ค่าเริ่มต้น 64)	$3n^2 \times 8$ B	ให้ชุดข้อมูลที่ใช้อยู่พอดี cache ลดการอ่านจาก RAM

ใน `blocked` แต่ละขั้นใช้ข้อมูล 3 tile (A, B, C) รวม $3 \times BS^2 \times 8$ ไบต์ เมื่อ BS = 32 จะได้ 24 KiB ซึ่งพอดี L1d 32 KiB และเมื่อ BS = 128 จะได้ 384 KiB ซึ่งยังอยู่ใน L2 512 KiB ตัวเลขนี้ใช้อธิบายผลการเลือก block size ในหัวข้อ 1.4

1.2 การออกแบบโปรแกรมและวิธีวัดผล

โปรแกรมแบ่งเป็น 4 ช่วง และมีเพียงช่วง kernel ที่ทำงานแบบขนาน การจับเวลาแต่ละช่วงแยกกันทำให้วัดสัดส่วน serial กับ parallel ได้ตรงจากการรันจริง แทนการเดาจากโค้ด



วาดโดยใช้ excalidraw

ลำดับการทำงานของ parallel.c · serial / parallel / serial

การแบ่งงาน ทุก kernel ใช้ `#pragma omp parallel for schedule(runtime)` ที่ลูปนอกสุด

(แถว i ของ C หรือกลุ่มแถวใน `blocked`) แต่ละช่องของ C จึงถูกเขียนโดย thread เดียว ไม่มี race condition และไม่ต้องใช้ lock หรือ atomic ส่วน A และ B ถูกอ่านอย่างเดียว `schedule(runtime)` ทำให้เปลี่ยน schedule และ chunk ได้จากตัวเลือก `-s`/`-c` โดยไม่ต้องคอมไพล์ใหม่

การจับเวลา

- ก่อนเริ่มจับเวลา โปรแกรมเปิด parallel region เปล่า 1 ครั้งเพื่อให้ OpenMP สร้าง thread team และอ่านจำนวน thread ที่ได้จริง ต้นทุนการสร้าง thread จึงไม่ปนอยู่ใน `t_mul`
- kernel ทำซ้ำ r รอบ และรายงานทั้งค่าที่ดีที่สุด (`t_mul_best`) และค่าเฉลี่ย (`t_mul_mean`) ในการทดลอง r ปรับตาม n: n ≤ 400 ทำ 10 รอบ, ≤ 800 ทำ 5 รอบ, ≤ 1600 ทำ 3 รอบ, 3200 ทำ 2 รอบ เพราะการรันที่สั้นมากได้รับผลจากสัญญาณรบกวนมากกว่า
- กราฟและตารางในรายงานใช้ `t_mul_best` เพราะใกล้เคียงความสามารถจริงของเครื่องที่สุด เมื่อมีโปรแกรมอื่นมาแย่ง CPU น้อยที่สุด

การตรวจความถูกต้อง เมตริกซ์ A และ B สร้างจาก LCG ที่มี seed คงที่ ทุกการรันจึงได้ข้อมูลเข้าเหมือนกัน ตัวเลือก `-v` เทียบผลกับการคำนวณ i-j-k แบบ serial (n ≤ 800 ตรวจทุกช่อง, n > 800 สุ่ม 2,000 ช่อง, ยอมรับ relative error ≤ 10^{-9})

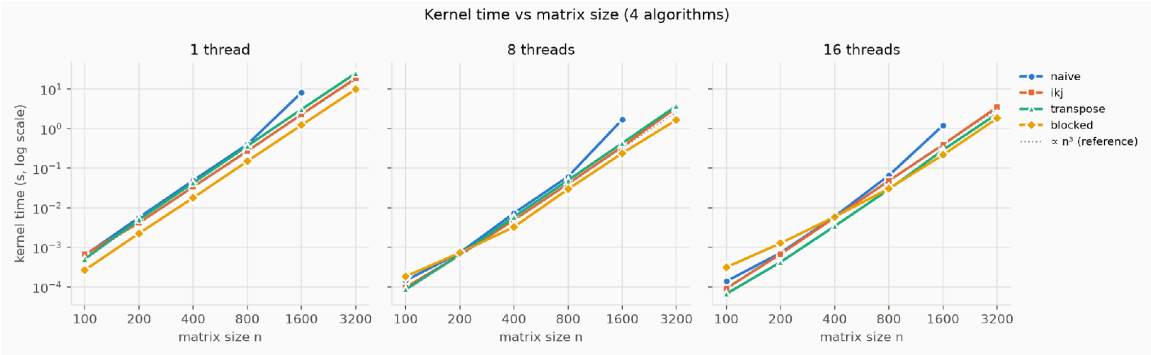
- ทดสอบด้วย `-v` ครบ 4 อัลกอริทึม × 1, 4, 16 threads ที่ n = 333 (ไม่ลงตัวกับ block 64) และ `blocked` n = 1600, BS = 48 ผ่านทุกกรณี ด้วย max relative error = 0
- ในข้อมูลการทดลองทั้งหมด (467 การรัน) ทุกการรันที่ใช้ n เดียวกันได้ checksum ตรงกันทุกหลัก ไม่ว่าจะใช้อัลกอริทึม จำนวน thread schedule หรือระดับ optimization ใด (เช่น n = 800 ได้ 1.2798002395×10^8 ทุกครั้ง)

ผลลัพธ์เท่ากันพอดีเพราะทุกอัลกอริทึมบวกผลคูณในแต่ละช่องตามลำดับ k = 0, 1, …, n−1 เหมือนกัน การแบ่งงานตามแถวไม่เปลี่ยนลำดับการบวกภายในช่องเดียวกัน ผลปัดเศษของ floating point จึงเหมือนกัน (กรณี -O3 -march=native คอมไพเลอร์อาจใช้คำสั่ง FMA ซึ่งปัดเศษต่างไปเล็กน้อย แต่ checksum 11 หลักที่พิมพ์ออกมายังตรงกัน)

1.3 ผลตามจำนวน thread และขนาด n (หัวข้อ 2.1, 2.2)

เมื่อ n ใหญ่พอ (n ≥ 400) speedup เพิ่มเกือบเป็นเส้นตรงจนถึง 8 threads (= จำนวน physical core) และได้ประมาณ 6 เท่า หลังจากนั้นแทบไม่เพิ่ม ส่วน n = 100 งานน้อยเกินไปจนการเพิ่ม thread ทำให้ช้าลง การทดลองใช้ n = 100, 200, 400, 800, 1600, 3200 × threads 1–8, 10, 12, 14, 16 × naive / ikj / blocked (naive ไม่รันที่ n = 3200 เพราะใช้เวลานานเกินไป)

เวลาเทียบกับ n



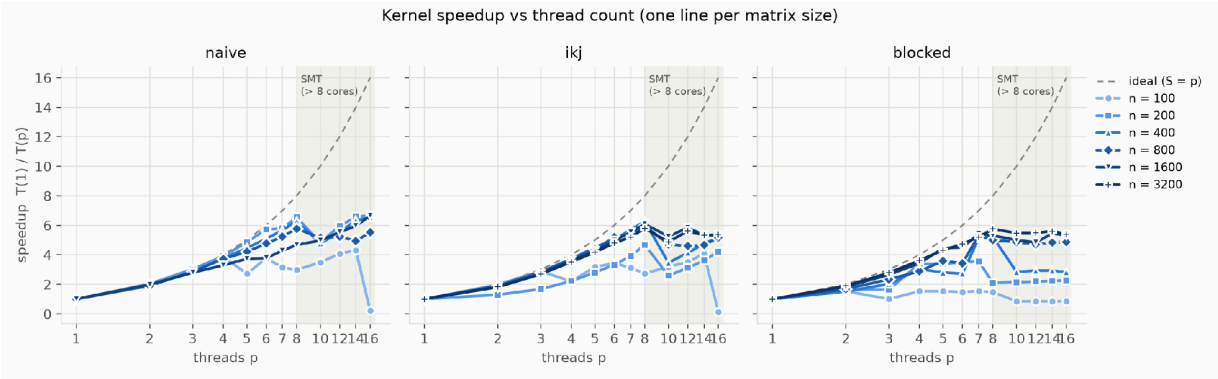
เส้นบนกราฟ log-log ขนานกับเส้นอ้างอิง ∝ n^3 เมื่อ n เพิ่ม 2 เท่า เวลาของ `ikj` ที่ 1 thread เพิ่ม 8.9 เท่า (800 → 1600: 0.259 → 2.294 s) และ 8.1 เท่า (1600 → 3200: → 18.66 s) ใกล้เคียง $2^3 = 8$ ตามทฤษฎี

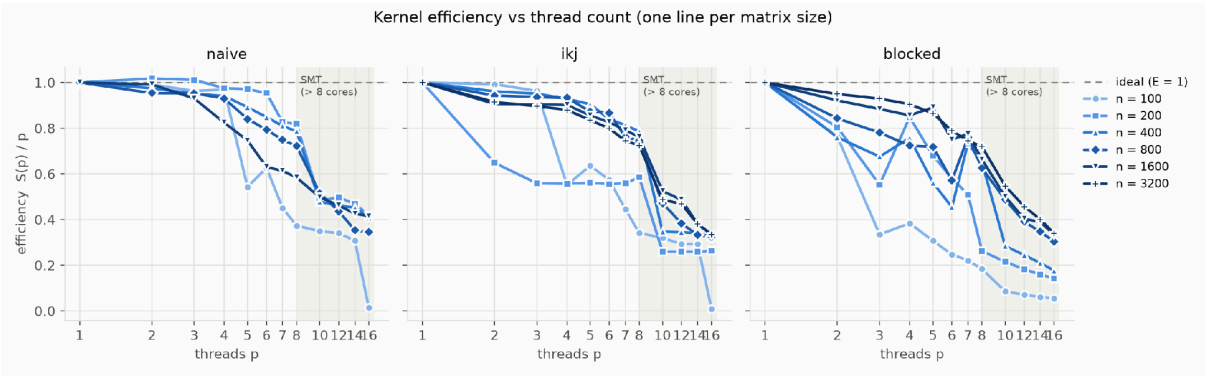
ข้อยกเว้นคือ `naive` ที่ n = 800 → 1600 เวลาเพิ่ม 21 เท่า (0.388 → 8.15 s) และความเร็วตกจาก 2.56 เหลือ 1.00 GFLOP/s เมื่อ n = 1600 เมตริกซ์ B มีขนาด 19.5 MiB ใหญ่กว่า L3 16 MiB การอ่าน B ตามคอลัมน์

(กระโดดทีละ 12.8 KB) จึงต้องไปอ่านจาก RAM เกือบทุกครั้ง นี่คือผลของ memory hierarchy ไม่ใช่ของจำนวนการคำนวณ

]เวลา speedup และ efficiency เทียบกับจำนวน thread

$$S(p) = \frac{1}{(1-f) + f/p}, \qquad S_{max} = \lim_{p\to\infty} S(p) = \frac{1}{1-f}, \qquad e(p) = \frac{1/S(p) - 1/p}{1 - 1/p}$$



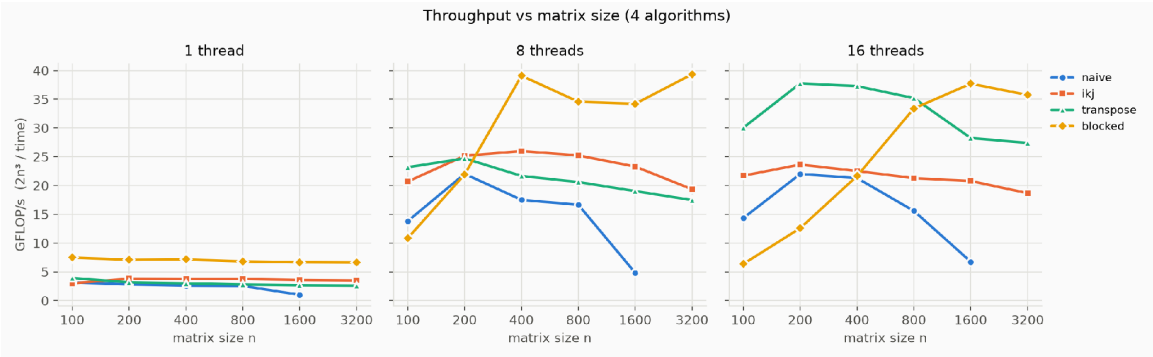


อัลกอริทึม	n	S(8)	E(8)	speedup สูงสุด
naive	400	6.29	0.79	6.53 ที่ p = 16
naive	1600	4.67	0.58	6.64 ที่ p = 16
ikj	1600	6.09	0.76	6.09 ที่ p = 8
ikj	3200	5.79	0.72	5.79 ที่ p = 8
blocked	3200	5.75	0.72	5.75 ที่ p = 8
ikj	100	2.73	0.34	4.09 ที่ p = 14
blocked	100	1.47	0.18	1.54 ที่ p = 7

ข้อสังเกตจากกราฟและตาราง

1. **จุดสูงสุดอยู่ที่ 8 threads** เท่ากับจำนวน physical core thread ที่ 9–16 เป็น SMT sibling ที่ใช้หน่วยคำนวณ floating point และ cache ร่วมกับ thread เดิม จึงไม่เพิ่มความเร็วให้งานที่คำนวณหนักอย่าง `ikj` และ `blocked`
2. **ที่ p = 10 speedup ตกลง** เพราะ static schedule แบ่งแถวเท่ากัน 10 ส่วน แต่ 2 core ต้องรับ 2 thread พร้อมกัน thread บน core ที่แชร์กันทำงานช้ากว่า และทุกคนต้องรอตัวที่ช้าที่สุดที่ barrier (load imbalance)
3. **`naive` ได้ประโยชน์จาก SMT** ที่ n = 1600 speedup เพิ่มจาก 4.67 (p = 8) เป็น 6.64 (p = 16) เพราะ naive ติด memory stall เป็นหลัก ขณะที่ thread หนึ่งรอข้อมูลจาก RAM อีก thread บน core เดียวกันใช้หน่วยคำนวณที่ว่างได้ (latency hiding)
4. **n = 100 ไม่คุ้มที่จะขนาน** kernel ที่ 1 thread ใช้เพียง 0.56 ms (`ikj`) ที่ 16 threads แต่ละ thread ได้งานแค่ 6–7 แถว ต้นทุน fork/join และการปลุก thread จึงมากกว่างาน speedup ของ naive และ ikj ตกเหลือประมาณ 0.2 (ช้ากว่า 1 thread)
5. **`blocked` ที่ n เล็กถูกจำกัดด้วยจำนวน block** เพราะขนานตามกลุ่มแถวขนาด 64 จำนวนงานที่แบ่งได้คือ $\lceil n/64 \rceil$ เท่านั้น n = 100 มี 2 block, n = 200 มี 4 block, n = 400 มี 7 block thread ที่เกินจำนวน block จึงไม่มีงานทำ ตรงกับที่ speedup สูงสุดของ n = 100 ไม่เกิน 1.54 และของ n = 200 ไม่เกิน 3.56

ความเร็วในหน่วย GFLOP/s

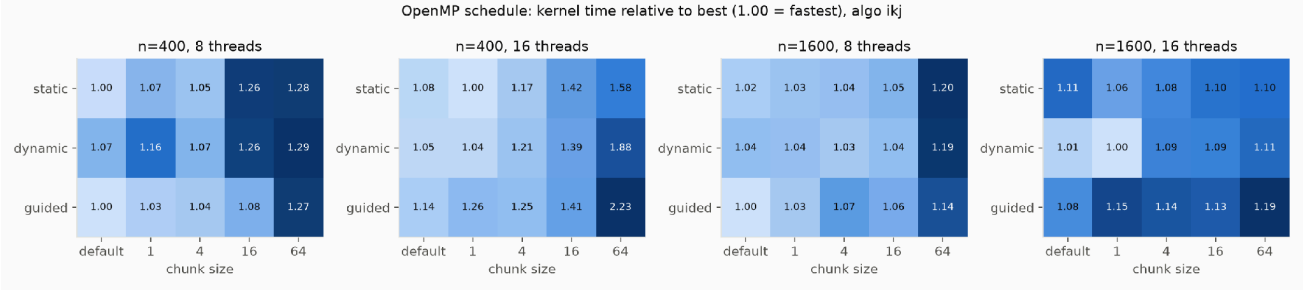


ที่ 1 thread `blocked` เร็วที่สุดทุก n (6.6–7.5 GFLOP/s) ประมาณ 2 เท่าของ `ikj` (3.5–3.8) ที่ 8 threads `blocked` ทำได้สูงสุด 39.3 GFLOP/s (n = 3200) ส่วน `transpose` เป็นอัลกอริทึมเดียวที่เร็วขึ้นชัดเจนจาก 8 ไป 16 threads (เช่น n = 1600: 19.0 → 28.3) เพราะ dot product มี dependency ต่อเนื่องผ่านตัวแปร `sum` ตัวเดียว หน่วยคำนวณจึงว่างให้ SMT sibling ใช้ได้

1.4 การตั้งค่าการทำงานของโปรแกรม (หัวข้อ 2.3)

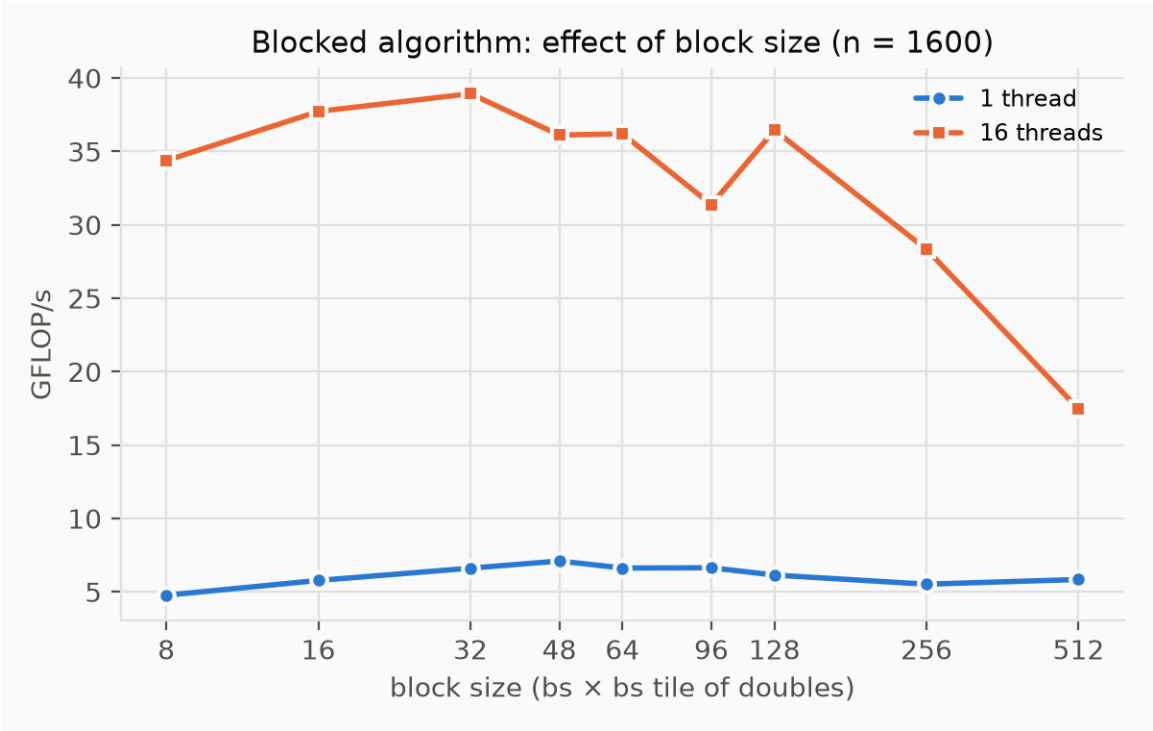
การตั้งค่าที่มีผลมากที่สุดคือระดับ optimization ของคอมไพเลอร์ (`-O3 -march=native` เร็วขึ้นสูงสุด 2.5 เท่า) รองลงมาคือ block size ส่วน OpenMP schedule มีผลน้อยเพราะทุกแถวมีงานเท่ากัน

OpenMP schedule × chunk size



- ที่ n = 1600, 8 threads ทุก schedule ต่างกันไม่เกิน ประมาณ 4% (21.4–22.4 GFLOP/s) เมื่อ chunk ≤ 16 เพราะงานของแต่ละแถวเท่ากัน (n คูณ-บวก × n) `static` จึงแบ่งงานได้สมดุลอยู่แล้ว และ `dynamic`/`guided` ไม่มีอะไรให้แก้
- chunk ใหญ่ทำให้ช้าลง: ที่ n = 400 chunk 64 แบ่งได้แค่ ⌈400/64⌉ = 7 ก้อน น้อยกว่าจำนวน thread ความเร็วตกจาก 25.7 เหลือ 20.2 GFLOP/s (8 threads) และ `guided` chunk 64 ที่ 16 threads เหลือเพียง 10.8 GFLOP/s
- `dynamic` chunk 1 ที่ n = 400 ช้ากว่า `static` เล็กน้อย (22.2 เทียบ 25.7) เพราะทุกแถวต้องของานจากคิวกลางด้วย atomic operation ต้นทุนนี้เห็นได้เมื่องานต่อแถวเล็ก

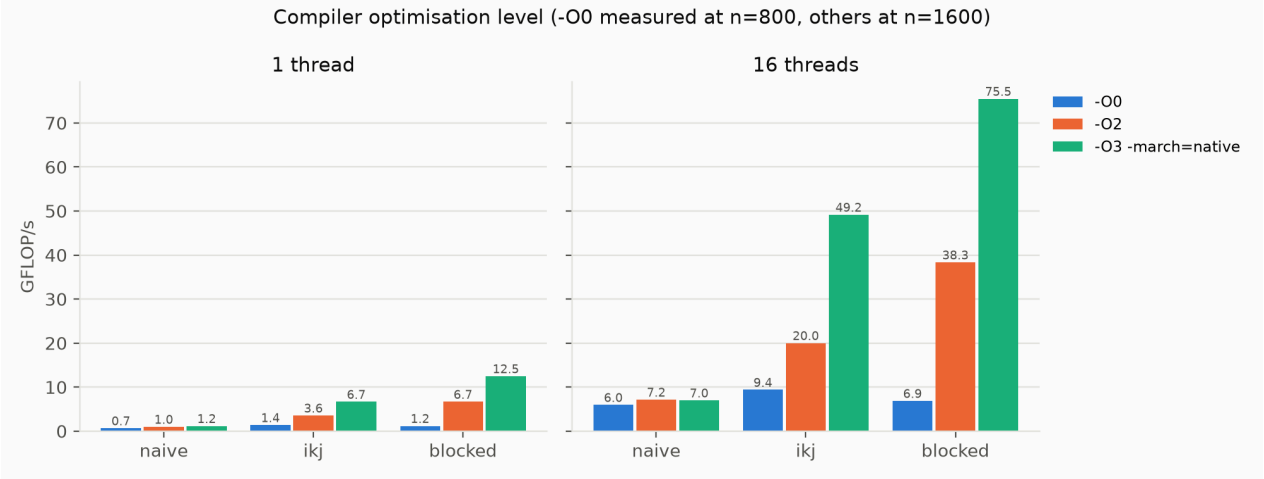
block size ของ `blocked` (n = 1600)



- ที่ 1 thread ดีที่สุดที่ BS = 48 (7.11 GFLOP/s) BS = 8 ช้าสุด (4.77) เพราะลูปในสุดสั้นเกินไปจนต้นทุนควบคุมลูปมากกว่างาน BS ≥ 256 ช้าลงเพราะ 3 tile รวมกัน (1.5 MiB) เกิน L2 512 KiB

- ที่ 16 threads ดีที่สุดที่ BS = 32 (38.9 GFLOP/s) แต่ BS = 512 เหลือ 17.5 เพราะมีเพียง ⌈1600/512⌉ = 4 block ให้แบ่ง มี thread ทำงานแค่ 4 ตัว block size จึงต้องเล็กพอทั้งต่อ cache และต่อจำนวนงานที่แบ่งให้ thread

ระดับ optimization ของคอมไพเลอร์



อัลกอริทึม (n = 1600)	-O2, 1 thr	-O3 native, 1 thr	-O2, 16 thr	-O3 native, 16 thr
naive	1.00	1.17	7.17	6.99
ikj	3.56	6.67	20.01	49.17
blocked	6.68	12.48	38.35	75.50

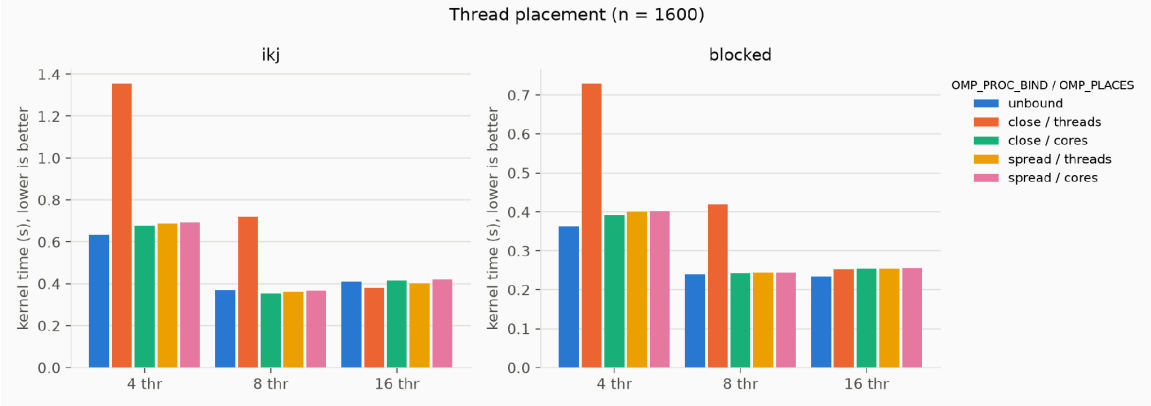
หน่วยเป็น GFLOP/s (`-O0` ทดสอบที่ n = 800 เพราะช้ามาก ได้ 0.66–1.38 GFLOP/s ที่ 1 thread)

`-march=native` อนุญาตให้ใช้คำสั่ง AVX2 (ทำ 4 double ต่อคำสั่ง) และ FMA ของ Zen 3 แทน SSE2 (2 double) ที่เป็นค่าเริ่มต้นของ x86-64 ลูปในของ `ikj` และ `blocked` เป็น stride 1 จึง vectorize ได้และเร็วขึ้นเกือบ 2 เท่า ส่วน `naive` แทบไม่เปลี่ยนเพราะติดที่การรอหน่วยความจำ และลูปในเป็นการบวกสะสมที่ gcc ไม่สลับลำดับให้หากไม่ใช้ `-ffast-math` ผลนี้แสดงว่าความเร็วจาก SIMD ภายใน core กับความเร็วจากหลาย thread คูณกันได้ (blocked: 6.68 → 75.5 GFLOP/s = 11.3 เท่า)

1.5 การตั้งค่าของระบบปฏิบัติการ (หัวข้อ 2.4)

ผลสำคัญคือ จำนวน thread ไม่ได้บอกความเร็ว จำนวน **physical core ที่ OS ให้ใช้จริง** ต่างหากที่บอก ทุกการทดลองในหัวข้อนี้ใช้ `ikj` (และ `blocked` ในการทดลอง binding) ที่ n = 1600 ยกเว้น wait policy ที่ใช้หลาย n และไม่มีการทดลองใดต้องใช้สิทธิ์ root

การวาง thread: `OMP_PROC_BIND` × `OMP_PLACES`



การตั้งค่า (ikj, n = 1600)	4 threads	8 threads	16 threads
ไม่ผูก (ให้ Linux scheduler เลือก)	12.95	22.14	19.94
close, places = threads	**6.04**	**11.38**	21.54
close, places = cores	12.13	23.31	19.77
spread, places = threads	11.90	22.66	20.41

หน่วยเป็น GFLOP/s การตั้ง `close` + `threads` ช้าลงครึ่งหนึ่งเพราะวาง 4 thread ไว้ที่ CPU 0–3 ซึ่งเป็นแค่ 2 physical core (CPU 0–1 และ 2–3 เป็น SMT sibling) ทุกคู่ต้องแย่งหน่วยคำนวณกัน เมื่อเปลี่ยนเป็น `places = cores` หรือ `spread` ความเร็วกลับมาเท่ากรณีไม่ผูก แสดงว่า scheduler ของ Linux กระจาย thread ไปคนละ core ได้ดีอยู่แล้วบนเครื่องที่มี NUMA node เดียว

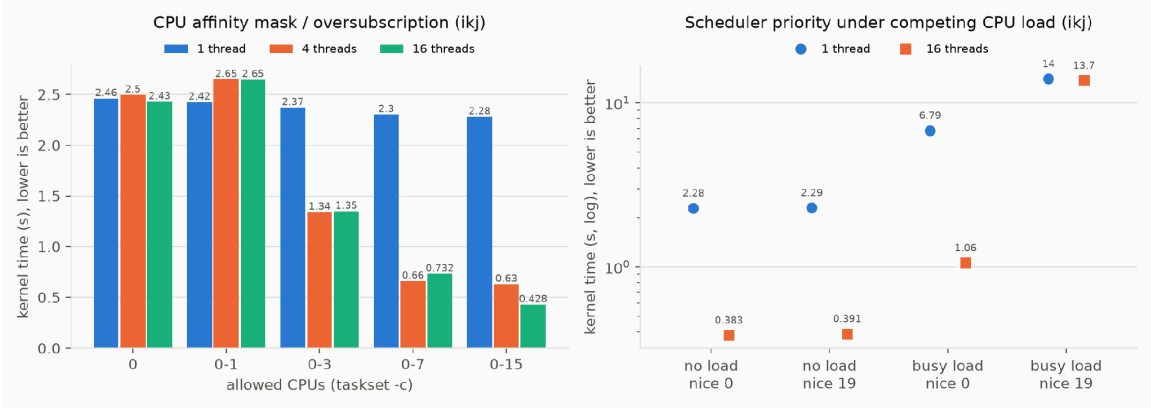
CPU affinity mask (`taskset`) และ oversubscription

CPU ที่อนุญาต	physical core	1 thread	4 threads	16 threads
0	1 (ครึ่ง core)	3.33	3.28	3.37
0–1	1	3.38	3.10	3.10
0–3	2	3.46	6.11	6.07
0–7	4	3.56	12.41	11.18
0–15	8	3.59	13.01	19.16

หน่วยเป็น GFLOP/s ความเร็วสูงสุดเพิ่มตามจำนวน physical core ที่อนุญาต (ประมาณ 3.1 GFLOP/s ต่อ core) ไม่ใช่จำนวน thread ที่ขอ CPU 0–1 ได้ความเร็วเท่า 1 thread เพราะเป็น core เดียวกัน เมื่อ thread มากกว่า CPU ที่อนุญาต (oversubscription) OS ต้องสลับ thread บน CPU เดิม ผลคือไม่เร็วขึ้นและช้าลงเล็กน้อย

(0–7: 12.41 → 11.18) จาก context switch และ cache ที่ถูกแทนที่

`nice` เมื่อมีโปรเซสอื่นแย่ง CPU



โหลดแข่งคือ busy loop 16 โปรเซส (เท่าจำนวน logical CPU) ที่ nice 0 จำกัดเวลาด้วย `timeout 600`

สถานการณ์	1 thread	16 threads
ไม่มีโหลด, nice 0	3.59	21.41
ไม่มีโหลด, nice 19	3.57	20.96
มีโหลด, nice 0	1.21	7.75
มีโหลด, nice 19	0.58	0.60

- เมื่อเครื่องว่าง `nice 19` ไม่มีผล เพราะ nice เป็นน้ำหนักสัมพัทธ์เทียบกับโปรเซสอื่น ไม่ใช่การจำกัด CPU
- เมื่อมีโหลด scheduler ของ Linux แบ่งเวลา CPU ตามน้ำหนักของ nice (nice 0 = 1024, nice 19 = 15 ตามตาราง `sched_prio_to_weight` ใน kernel) thread ที่ nice 19 จึงได้ CPU เพียงประมาณ 15/(1024+15) ≈ 1.4% เมื่อแชร์ CPU กับ busy loop 1 ตัว
- ที่ nice 19 ใช้ 16 threads ก็ไม่เร็วกว่า 1 thread (0.60 เทียบ 0.58) เพราะ static schedule ต้องรอ thread ที่ช้าที่สุด และทุก thread ต่างถูกแย่ง CPU เท่าๆ กัน
- ที่ nice 0 กับ 1 thread ความเร็วเหลือ 1/3 แม้ควรได้ครึ่งหนึ่งของ CPU สาเหตุที่เป็นไปได้คือ busy loop บน SMT sibling แย่งหน่วยคำนวณของ core เดียวกัน และความถี่ boost ลดลงเมื่อทุก core ทำงาน (ข้อนี้เป็นการอนุมาน ไม่ได้วัดความถี่ CPU ระหว่างการทดลอง)

`OMP_WAIT_POLICY` (16 threads)

n	active (spin)	passive (sleep)
100	18.48	15.81
400	22.00	23.25
1600	18.55	19.55

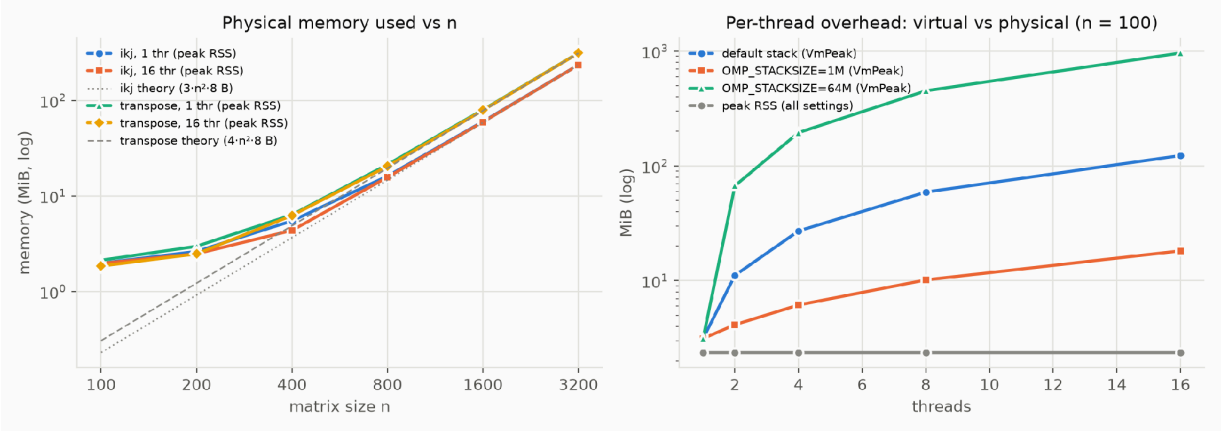
หน่วยเป็น GFLOP/s ที่ n = 100 แบบ active เร็วกว่า 17% เพราะ thread ที่รอจะวนเช็คงานอยู่และเริ่มทำงานได้ทันที ส่วน passive ต้องให้ OS ปลุก thread ผ่าน futex ซึ่งมี latency ระดับไมโครวินาที เทียบได้กับ kernel ที่

ใช้เวลาเพียง 0.1 ms เมื่อ n ใหญ่ passive เร็วกว่าเล็กน้อย (ประมาณ 5% ซึ่งใกล้เคียงระดับสัญญาณรบกวน) อาจเพราะ thread ที่เสร็จก่อนไม่ได้ spin แย่ง core กับ SMT sibling ที่ยังทำงานอยู่

1.6 หน่วยความจำที่ต้องใช้ (หัวข้อ 2.5)

หน่วยความจำกายภาพ (RSS) ขึ้นกับ n เกือบทั้งหมดและแทบไม่ขึ้นกับจำนวน thread ส่วนหน่วยความจำเสมือน (VmPeak) เพิ่ม 8 MiB ต่อ thread จาก stack ที่จองไว้แต่ไม่ได้ใช้ ค่า RSS อ่านจาก `getrusage()` และตรวจไขว้กับ `/usr/bin/time` (ต่างกันไม่เกิน 0.3 MiB) ส่วน VmPeak อ่านจาก `/proc/self/status`

$$M_{theory} = k \cdot n^2 \cdot 8 \text{ bytes}, \qquad k = 3 \text{ (A, B, C)},\ k = 4 \text{ (transpose: + B}^T\text{)}$$



อัลกอริทึม	n	ทฤษฎี (MiB)	RSS 1 thr	RSS 16 thr	VmPeak 1 thr	VmPeak 16 thr
ikj	100	0.23	1.97	1.97	3.1	123.2
ikj	800	14.65	16.15	15.71	17.6	137.7
ikj	3200	234.38	235.59	235.37	237.4	357.4
transpose	3200	312.50	313.87	313.46	315.5	435.6

ข้อสังเกต

1. **RSS = ทฤษฎี + ประมาณ 1.0–1.7 MiB** ส่วนที่เกินคือโค้ดของโปรแกรม libc libgomp และ stack ที่ใช้จริง เมื่อ n เล็ก (100) ส่วนนี้มากกว่าเมตริกซ์เองเกือบ 8 เท่า เมื่อ n ใหญ่จะเหลือไม่ถึง 1%
2. **การเพิ่ม thread ไม่เพิ่ม RSS** เพราะทุก thread ใช้เมตริกซ์ชุดเดียวกันใน address space เดียวกัน (shared memory model) ไม่มีการคัดลอกข้อมูลเหมือน MPI
3. **VmPeak เพิ่ม 8 MiB ต่อ thread พอดี** (n = 100: 3.1 + 15 × 8 = 123.1 MiB) แต่ละ worker thread ได้ stack ขนาดเท่า `ulimit -s` = 8 MiB ที่ map ไว้ด้วย `mmap()` แต่ kernel จะให้หน้าจริงเมื่อถูกแตะครั้งแรกเท่านั้น (demand paging) RSS จึงไม่เปลี่ยน
4. **`transpose` ใช้หน่วยความจำมากกว่า 4/3 เท่า** ตามทฤษฎี (313.9 เทียบ 235.6 MiB ที่ n = 3200) แต่ไม่ได้เร็วกว่า `blocked` ที่ใช้หน่วยความจำเท่า `ikj`

overhead ต่อ thread ตาม `OMP_STACKSIZE` (n = 100)

OMP_STACKSIZE	VmPeak 1 thr	VmPeak 16 thr	เพิ่มต่อ thread	RSS (ทุกกรณี)
default	3.1 MiB	123.2 MiB	8 MiB	2.37 MiB
1M	3.1 MiB	18.2 MiB	1 MiB	2.37 MiB
64M	3.1 MiB	963.2 MiB	64 MiB	2.37 MiB

ที่ `OMP_STACKSIZE=64M` กับ 16 threads โปรเซสจอง address space เกือบ 1 GiB แต่ใช้ RAM จริงเพียง 2.37 MiB เท่าเดิม นี่คือเหตุผลที่การดูหน่วยความจำต้องแยก virtual กับ physical และเป็นจุดเชื่อมกับเครื่องมือในส่วนที่ 2 ซึ่งแสดงทั้งสองค่าแยกกันในแต่ละ region

โปรแกรมจำกัด n ≤ 4096 (3 เมตริกซ์ = 384 MiB, transpose = 512 MiB) และจำนวน thread ≤ 4 × จำนวน CPU เพื่อไม่ให้ใช้หน่วยความจำจนระบบต้อง swap หรือเกิด OOM (เงื่อนไข DO NO HARM)

1.7 สัดส่วน parallel vs serial ตาม Amdahl's Law

สำหรับ `ikj` ที่ n = 1600 โค้ดส่วน parallel ใช้เวลา 98.9% ของการรัน 1 thread แต่ speedup ที่วัดได้จริงเท่ากับโปรแกรมที่มีส่วน parallel เพียงประมาณ **94% parallel / 6% serial** ช่องว่าง 5% นี้ไม่ใช่โค้ด serial แต่เป็น overhead ของการทำงานขนานที่ทำตัวเหมือนส่วน serial

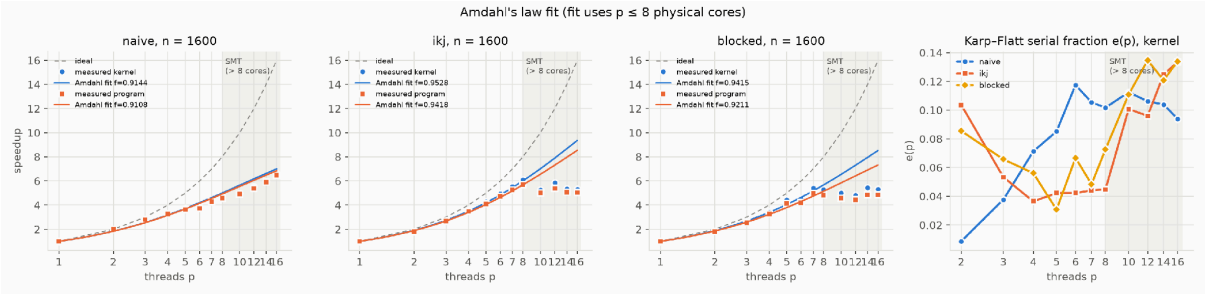
$$S(p) = \frac{1}{(1-f) + f/p}, \qquad S_{max} = \lim_{p\to\infty} S(p) = \frac{1}{1-f}, \qquad e(p) = \frac{1/S(p) - 1/p}{1 - 1/p}$$

f คือสัดส่วนที่ทำขนานได้ และ e(p) คือ Karp–Flatt metric ซึ่งคำนวณสัดส่วน serial ย้อนกลับจาก speedup ที่วัดได้ที่ p แต่ละค่า

ประมาณ f ด้วย 3 วิธีจากข้อมูลที่วัดจริงใน `scaling.csv`

- **f_phase** = t_mul / t_total ที่ 1 thread คือสัดส่วนของโค้ดที่ "ออกแบบให้ขนาน"
- **f_kernel** = fit สมการ Amdahl แบบ least squares กับ speedup ของ kernel อย่างเดียว
- **f_program** = fit แบบเดียวกับเวลาทั้งโปรแกรม (init + kernel 1 รอบ + checksum)

การ fit ใช้เฉพาะ 2 ≤ p ≤ 8 เพราะ Amdahl สมมติว่าหน่วยประมวลผลทุกตัวเท่ากัน thread ที่ 9–16 เป็น SMT ที่ไม่ได้เพิ่มหน่วยคำนวณจริง จึงไม่เข้าเงื่อนไขนี้



อัลกอริทึม	n	f_phase	f_kernel	f_program	e(8)	S_max (program)
ikj	100	0.852	0.824	0.541	0.276	2.2
ikj	400	0.944	0.966	0.916	0.039	11.9
ikj	1600	0.989	0.953	0.942	0.045	17.2
ikj	3200	0.995	0.945	0.940	0.054	16.5
naive	1600	0.996	0.914	0.911	0.102	11.2
blocked	1600	0.979	0.942	0.921	0.073	12.7
blocked	3200	0.990	0.952	0.943	0.056	17.6

ตัวอย่างการคำนวณ `ikj` n = 1600 วัดได้ S(8) = 6.09 จะได้ e(8) = (1/6.09 − 1/8) / (1 − 1/8) = (0.1642 − 0.1250) / 0.875 = 0.045 หมายความว่าพฤติกรรมที่วัดได้เท่ากับโปรแกรมที่มีส่วน serial 4.5%

การตีความ

1. **ส่วน serial ตามโค้ดลดลงตาม 1/n** เพราะ init และ checksum เป็น O(n²) ส่วน kernel เป็น O(n³) เมื่อ n เพิ่ม 2 เท่า สัดส่วน serial ของ `ikj` ลดจาก 1.15% เหลือ 0.54% (n = 1600 → 3200) ปัญหาใหญ่จึงขนานได้ดีกว่า สอดคล้องกับมุมมองของ Gustafson's Law
2. **f_kernel < 1 ทั้งที่ kernel ไม่มีโค้ด serial** ส่วน "serial แฝง" 5–9% มาจาก (ก) memory bandwidth และ L3 ที่ทุก core ใช้ร่วมกัน (ข) fork/join และ barrier ตอนจบ parallel region (ค) ความถี่ boost ที่ลดลงเมื่อหลาย core ทำงานพร้อมกัน Amdahl's Law เดิมไม่ได้คำนึงถึงปัจจัยเหล่านี้
3. **e(p) บอกชนิดของ overhead** `ikj` และ `blocked` มี e(p) คงที่ประมาณ 0.04–0.07 ช่วง p = 4–8 แปลว่า overhead ทำตัวเหมือนส่วน serial คงที่ ส่วน `naive` มี e(p) เพิ่มตาม p (0.01 → 0.12) แปลว่า overhead โตตามจำนวน thread ซึ่งตรงกับการแย่ง memory bandwidth ของ naive ที่อ่าน RAM มากที่สุด
4. **เส้น fit คาดการณ์เกินจริงเมื่อ p > 8** เช่น `ikj` ทำนาย S(16) ≈ 9.4 แต่วัดได้ประมาณ 5.3 เพราะสมมติฐานที่ว่า thread ทุกตัวเป็นหน่วยประมวลผลเต็มตัวไม่เป็นจริงสำหรับ SMT
5. **n เล็ก overhead ชนะ** ที่ n = 100 f_program = 0.54 เท่ากับ speedup สูงสุดทางทฤษฎีเพียง 2.2 เท่า ไม่ว่าจะใช้กี่ core

สรุป Intro to Parallel Programming

- โปรแกรมที่เขียนมีสัดส่วน parallel : serial ประมาณ **99 : 1 ตามโค้ด** และ **94 : 6 ตามพฤติกรรมที่วัดได้** (`ikj`, n = 1600) ทำให้ speedup สูงสุดทางทฤษฎีประมาณ 17 เท่า บนเครื่องนี้ได้จริง 6.1 เท่าที่ 8 core
- ปัจจัยที่มีผลมากที่สุดเรียงตามลำดับ: จำนวน physical core ที่ใช้ได้จริง > การเข้าถึงหน่วยความจำแบบเป็นมิตรกับ cache (blocked เร็วกว่า naive 6.6 เท่าที่ 1 thread, n = 1600) > SIMD จาก `-march=native` > schedule และ wait policy
- การตั้งค่า OS ที่ผิด เช่น ผูก thread ไว้บน SMT sibling หรือรันที่ `nice 19` ตอนเครื่องมีโหลด ทำให้เสียความเร็วได้ครึ่งหนึ่งจนถึง 36 เท่า
- หน่วยความจำกายภาพขึ้นกับ n² เท่านั้น การเพิ่ม thread เพิ่มเฉพาะ virtual memory 8 MiB ต่อ thread

ส่วนที่ 2 — Memory Map Visualization

2.1 ภาพรวมและแหล่งข้อมูล

`mmap` เป็นเครื่องมือบน Linux ที่แสดงภาพ address space ของโปรเซสหนึ่งตัว ว่ามี region อะไรบ้าง แต่ละ region ใหญ่เท่าไร อยู่ใน RAM จริงเท่าไร และเปลี่ยนแปลงอย่างไรตามเวลาโดยะเลือกใช้ข้อมูลขาเข้าแบบที่ 1 ของโจทย์ คือ memory map ที่ OS เปิดเผยผ่าน procfs และแสดงผลได้ทั้งแบบเวลาจริง (real time) และแบบ snapshot เขียนด้วย Rust เพราะได้ binary ไฟล์เดียวที่เร็ว ปลอดภัยด้านหน่วยความจำ และมีไลบรารีวาด TUI (`ratatui`) ที่ใช้งานง่าย

ไฟล์ใน `/proc`	ให้ข้อมูลอะไร	ใช้ทำอะไรในโปรแกรม
`/proc/PID/maps`	1 บรรทัดต่อ 1 region (VMA): ช่วงที่อยู่ สิทธิ์ offset device inode path	รายการ region (ใช้เมื่ออ่าน smaps ไม่ได้)
`/proc/PID/smaps`	เหมือน maps + ตัวนับต่อ region เช่น Rss, Pss, Private_Dirty, Swap	แผนที่ส่วนที่อยู่ใน RAM, ตาราง region, สรุปตามชนิด
`/proc/PID/status`	ค่ารวมทั้งโปรเซส: VmSize, VmRSS, VmHWM, RssAnon, RssFile, VmSwap, Threads	แถบ memory และกราฟ RSS ย้อนหลัง
`/proc/PID/exe`, `cmdline`	path ของตัวโปรแกรมและคำสั่งที่ใช้รัน	แยก region ชนิด `exe` และแสดงชื่อโปรเซส
`/proc/meminfo`	หน่วยความจำทั้งเครื่อง	เทียบว่าโปรเซสใช้ RAM กี่ % ของเครื่อง

ตัวอย่างข้อมูลดิบจากเครื่องที่ทดสอบ (ตัดมาบางส่วน) บรรทัดแรกคือ region หนึ่งใน `maps` ส่วนกลุ่มสองคือ region `[stack]` ใน `smaps` ที่มีตัวนับต่อท้าย

```
558bd7300000-558bd7306000 r-xp 00002000 103:02 13252906    /usr/bin/head

7ffcfe231000-7ffcfe253000 rw-p 00000000 00:00 0            [stack]
Size:                136 kB
Rss:                  20 kB
Pss:                  20 kB
Private_Dirty:        20 kB
Anonymous:            20 kB
```

stack นี้ map ไว้ 136 KiB แต่อยู่ใน RAM จริงเพียง 20 KiB ความต่างระหว่าง "ขนาดที่ map" กับ "ส่วนที่อยู่ใน RAM" นี้คือสิ่งที่เครื่องมือเน้นแสดง เพราะเป็นผลโดยตรงของ demand paging และเป็นปรากฏการณ์เดียวกับ VmPeak กับ RSS ในหัวข้อ 1.6

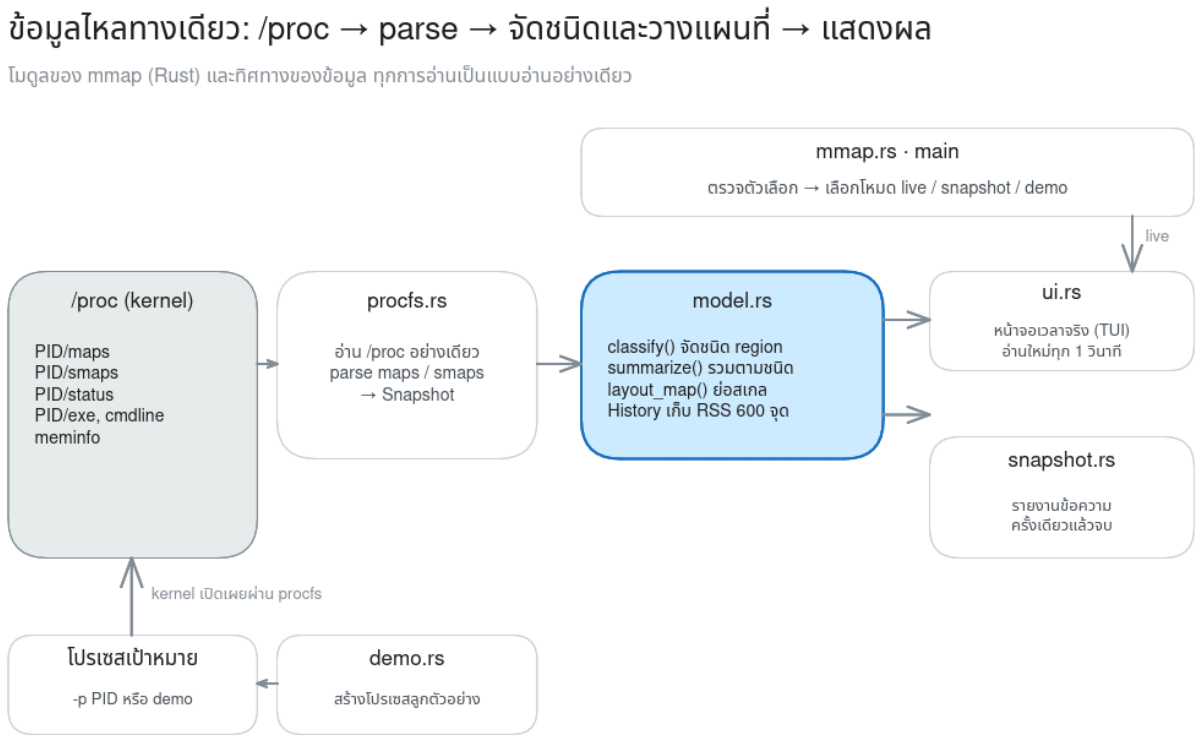
คำศัพท์ที่ใช้

- **VSZ / Size** ขนาด address ที่ map ไว้ (virtual) ไม่ได้หมายความว่าใช้ RAM
- **RSS** จำนวนหน้าที่อยู่ใน RAM จริง นับหน้าที่แชร์กับโปรเซสอื่นเต็มจำนวน

- **PSS** RSS ที่หารหน้าที่แชร์ด้วยจำนวนโปรเซสที่ใช้ร่วมกัน เช่น libc ที่ทุกโปรเซสใช้ จึงเหมาะกับคำถาม "โปรเซสนี้ใช้ RAM จริงเท่าไร"
- **Dirty** หน้าที่ถูกเขียนแล้ว ถ้าต้องคืน RAM ต้องเขียนกลับไฟล์หรือ swap ก่อน ส่วน **Swap** คือหน้าที่ถูกย้ายออกไปอยู่ใน swap แล้ว

2.2 การพัฒนาโปรแกรม

โปรแกรมแบ่งเป็น 6 โมดูล (รวม 2,513 บรรทัด) โดยแยก "การอ่านข้อมูล" "การตีความ" และ "การแสดงผล" ออกจากกัน ทำให้หน้าจอเวลาจริงและรายงาน snapshot ใช้ตรรกะชุดเดียวกัน และทดสอบส่วน parse ได้ด้วย unit test โดยไม่ต้องเปิดหน้าจอ



วาดโดยใช้ Excalidraw

สถาปัตยกรรมของ mmap · 6 โมดูล

ไฟล์	บรรทัด	หน้าที่
`mmap.rs`	148	ตรวจตัวเลือกทุกตัวพร้อมช่วงค่าที่อนุญาต และเลือกโหมดการทำงาน
`procfs.rs`	287	อ่านและ parse ไฟล์ใน `/proc` เป็นโครงสร้าง `Snapshot` และจัดการกรณีอ่านไม่ได้

ไฟล์	บรรทัด	หน้าที่
`model.rs`	542	จัดชนิด region, สรุปตามชนิด, คำนวณการวางแผนที่, เก็บประวัติ RSS
`ui.rs`	1,056	หน้าจอเต็มจอแบบเวลาจริง คีย์บอร์ด เมาส์ หน้าเลือกโปรเซส
`snapshot.rs`	229	รายงานข้อความครั้งเดียว มีหรือไม่มีสี ANSI ก็ได้
`demo.rs`	251	โปรเซสตัวอย่างที่จอง/คืนหน่วยความจำเป็นจังหวะ

ลำดับการพัฒนา

1. **Parser ก่อน** เขียน `parse_maps_line`, `parse_smaps`, `parse_status` ตามรูปแบบใน man page `proc_pid_maps(5)` และเอกสาร kernel `filesystems/proc.rst` path คือทุกอย่างหลัง inode เพื่อรองรับชื่อไฟล์ที่มีช่องว่าง
2. **ตรวจกับ kernel** unit test `snapshot_of_self_matches_kernel_totals` อ่าน smaps ของตัวเอง แล้วเทียบผลรวมกับค่าใน status เพื่อยืนยันว่า parse ครบทุก region
3. **จัดชนิดและสรุป** (`model.rs`) แล้วทำรายงาน snapshot ก่อน เพราะตรวจผลได้ง่ายเป็นข้อความ
4. **หน้าจอเวลาจริง** ด้วย `ratatui` + `crossterm` สไตล์ btop
5. **โปรเซส demo** เพื่อให้มีสิ่งที่เปลี่ยนแปลงให้ดู และใช้อธิบายแนวคิด OS ทีละเรื่อง (หัวข้อ 2.3)

การจัดชนิด region

kernel บอกแค่ช่วงที่อยู่ สิทธิ์ และ path โปรแกรมต้องตีความเองว่าแต่ละ region คืออะไร โดยตรวจเงื่อนไขตามลำดับในตาราง (ข้อแรกที่ตรงคือผล)

ชนิด	อักษรใน snapshot	เงื่อนไข	ความหมายทาง OS
heap	H	path = `[heap]`	พื้นที่ที่ขยายด้วย `brk()`
stack	S	path ขึ้นต้น `[stack`	stack ของ main thread
kernel	K	`[vdso]`, `[vvar]`, `[vsyscall]`	หน้าที่ kernel map ให้เพื่อเรียก syscall บางตัวได้เร็ว
shm	M	`/dev/shm/`, `memfd:`, SYSV หรือ anonymous ที่สิทธิ์ลงท้าย `s`	shared memory ระหว่างโปรเซส
exe	E	path ตรงกับ `/proc/PID/exe`	text, data, bss ของตัวโปรแกรม
lib	L	ชื่อไฟล์ลงท้าย `.so` หรือมี `.so.`	shared library
file	F	มี path อื่น	ไฟล์ที่ถูก `mmap()`

ชนิด	อักษรใน snapshot	เงื่อนไข	ความหมายทาง OS
guard	G	ไม่มี path และสิทธิ์ `---`	หน้าที่ห้ามเข้าถึง (guard page หรือพื้นที่จองไว้)
thr-stack	T	anonymous ≥ 16 KiB ที่ต่อจาก guard ≤ 64 KiB	stack ของ thread อื่น (อนุมานจากตำแหน่ง)
anon	A	ไม่ตรงข้อใดเลย	malloc ก้อนใหญ่ หรือ `mmap()` แบบ anonymous

กฎ `thr-stack` มาจากวิธีที่ glibc สร้าง thread: map stack ด้วย `mmap()` แล้วตั้งหน้าต่ำสุดเป็น `PROT_NONE` (guard page) เพื่อจับ stack overflow เกณฑ์ 16 KiB เพิ่มเข้ามาหลังพบว่า Rust สร้าง alternate signal stack ขนาด 8 KiB ต่อจาก guard page เหมือนกัน ซึ่งไม่ใช่ thread stack

การย่อสเกลแผนที่ address space

address space ฝั่ง user ของ x86-64 กว้าง 128 TiB แต่โปรเซสทั่วไป map ไว้ไม่กี่สิบ MiB กระจายอยู่สองฝั่ง (ตัวโปรแกรม+heap ใกล้ 0x55… และ library+stack ใกล้ 0x7f…) ถ้าวาดตามสัดส่วนจริงทุก region จะกลายเป็นจุดที่มองไม่เห็น `layout_map()` จึงให้ความกว้างตามสูตร

$$w_{region} = \log_2\left(\frac{\text{size}}{4\,\text{KiB}}\right) + 1, \qquad w_{gap} = 0.5,$$

$$\text{cells}_{resident} = \text{round}\left(\frac{\text{Rss}}{\text{Size}} \times \text{cells}_{region}\right)$$

แล้วกระจายน้ำหนักรวมลงบนจำนวนช่องที่มี แต่ละช่องแสดง region ที่ครอบคลุมมากที่สุด region 4 KiB ได้น้ำหนัก 1 ส่วน region 64 MiB ได้ 15 จึงเห็นทั้งคู่ ช่องแรกๆ ของแต่ละ region ถูกระบายสีสดตามสัดส่วน Rss/Size ทำให้เห็น demand paging ได้ด้วยตา ข้อแลกคือความกว้างบนแผนที่ไม่ใช่ขนาดจริง โปรแกรมจึงแสดงขนาดจริงไว้ในตารางคู่กันเสมอ

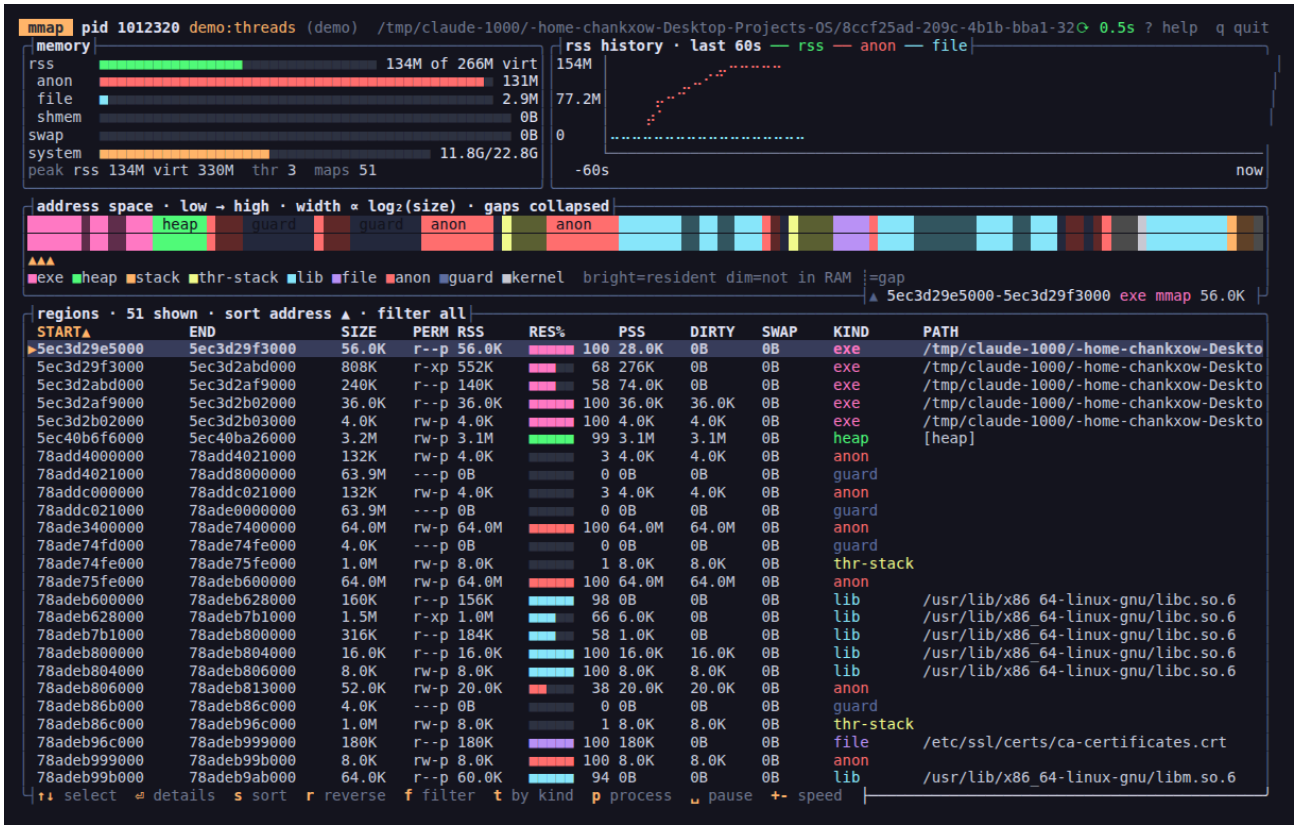
การจัดการกรณีผิดปกติ

- อ่าน `smaps` ไม่ได้ → ใช้ `maps` แทน แสดง region โดยไม่มี RSS/PSS
- โปรเซสของผู้ใช้อื่น (permission denied) → แสดงเฉพาะค่าจาก `status` พร้อมข้อความเตือน เพราะ kernel อนุญาตให้อ่าน maps เฉพาะเจ้าของหรือผู้ที่มีสิทธิ์ ptrace
- โปรเซสจบการทำงานระหว่างดู → คงข้อมูลล่าสุดไว้และให้กด `p` เลือกโปรเซสใหม่
- kernel thread เช่น `kthreadd` ไม่มี mapping ฝั่ง user → แจ้งว่าไม่มี mapping

2.3 ความสามารถของโปรแกรม

โปรแกรมมี 3 โหมด คือหน้าจอเวลาจริง, รายงาน snapshot และโปรเซส demo ภาพทั้งหมดในหัวข้อนี้จับจากการรันจริงของ `./mmap --demo -i 500` บนเทอร์มินัลขนาด 140 × 44 ตัวอักษร

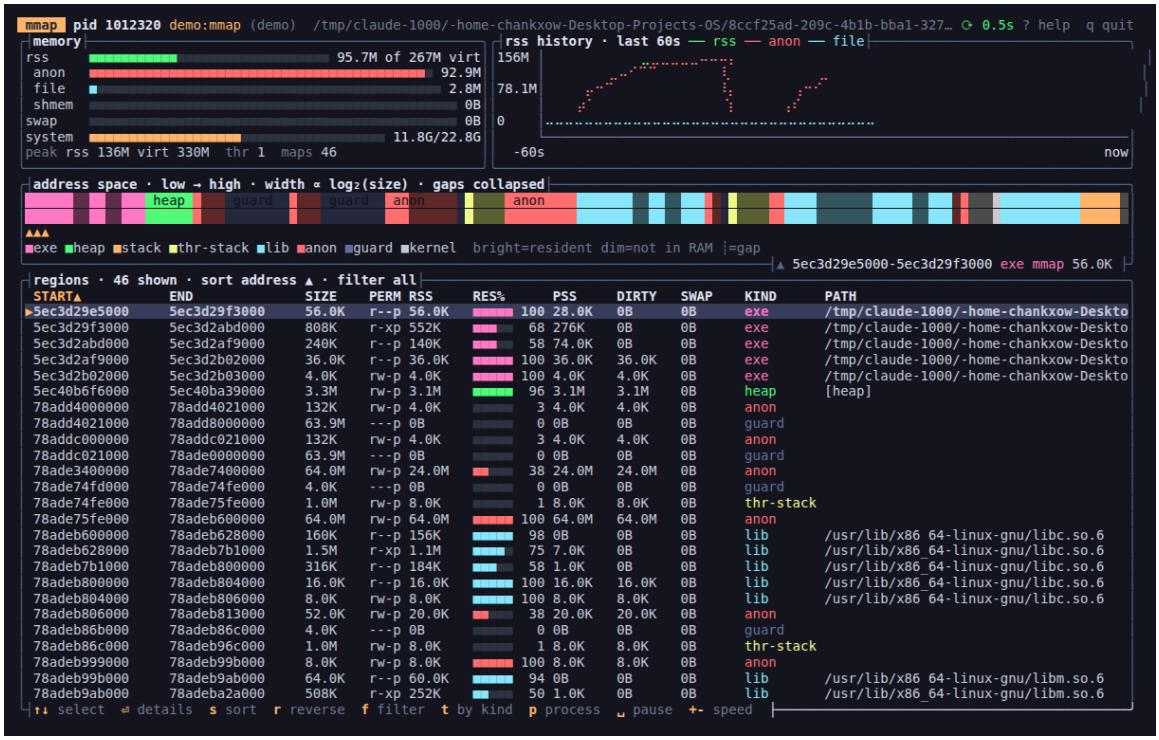
หน้าจอเวลาจริง



หน้าจอแบ่งเป็น 4 ส่วนจากบนลงล่าง

1. **memory** (ซ้ายบน) แถบ RSS เทียบกับ virtual, RSS แยก anon / file / shmem, swap, หน่วยความจำทั้งเครื่อง และค่าสูงสุด (peak rss / virt) จำนวน thread และจำนวน region
2. **rss history** (ขวาบน) กราฟ RSS, anon, file ย้อนหลัง 60 วินาที
3. **address space** แผนที่ทุก region เรียงจาก address ต่ำไปสูง สีตามชนิด แถบบนสีสด = ส่วนที่อยู่ใน RAM แถบล่างสีทึบ = map ไว้แต่ไม่อยู่ใน RAM และ ▲ ชี้ region ที่เลือก
4. **regions** ตาราง region พร้อมที่อยู่ ขนาด สิทธิ์ RSS แท่ง % ที่อยู่ใน RAM, PSS, dirty, swap, ชนิด และ path

สิ่งที่เห็นได้จากภาพนี้: thread ใหม่ 2 ตัวได้ `thr-stack` ขนาด 1.0 MiB คนละก้อน ต่อจาก `guard` 4 KiB แต่อยู่ใน RAM เพียง 8 KiB (1%) เพราะ thread แตะ stack แค่นั้น นอกจากนี้ยังเกิด `guard` ขนาด 63.9 MiB สองก้อน ต่อจาก anon 132 KiB ซึ่งคือ malloc arena ของแต่ละ thread ที่ glibc จอง address ไว้ 64 MiB ทำให้ peak virt กระโดดเป็น 330 MiB ทั้งที่ RSS เพิ่มแทบไม่ได้

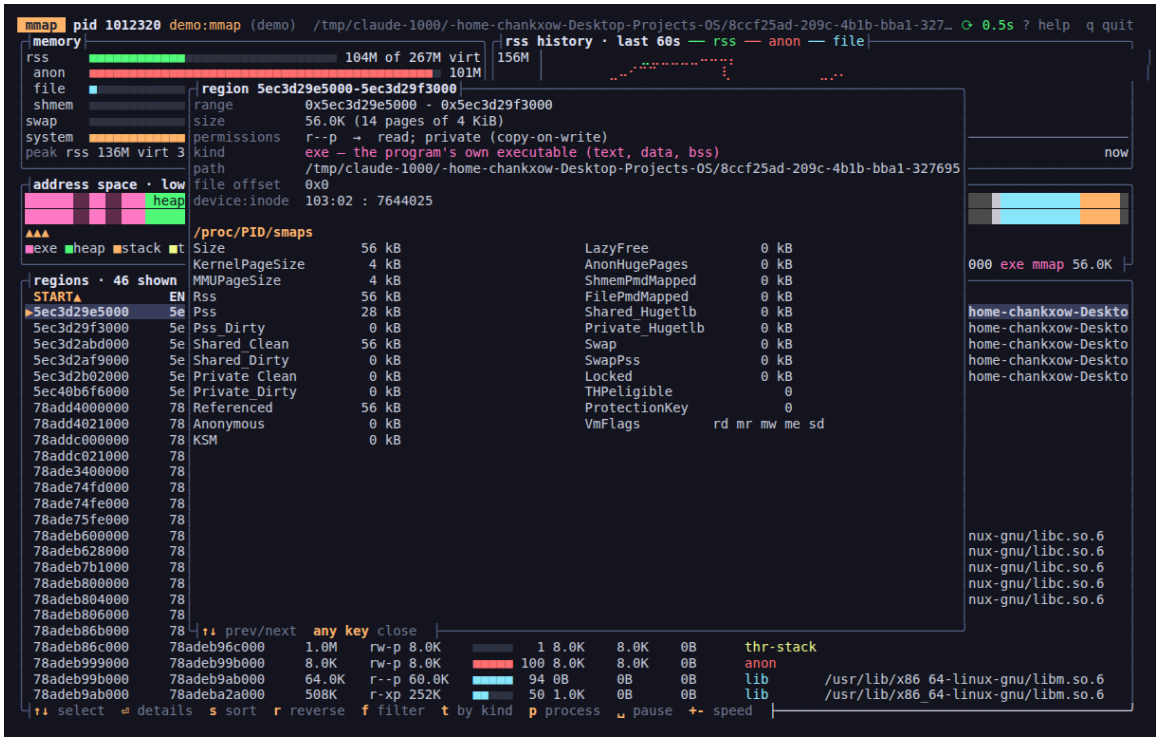


ภาพนี้จับหลังจากนั้น 12 วินาที กราฟ rss history แสดงวงจรครบรอบ: RSS เพิ่มเป็นขั้นบันได ตกลงเกือบศูนย์ตอน `demo:free` แล้วเริ่มเพิ่มใหม่ region `anon` ขนาด 64.0 MiB ที่เพิ่ง `mmap()` มี RSS เพียง 24.0 MiB (38%) เพราะ demo เขียนทีละ 8 MiB และ kernel จะให้หน้าจริงเมื่อเกิด page fault ครั้งแรกเท่านั้น (demand paging) ส่วน region `thr-stack` ยังคงอยู่แม้ thread จบไปแล้ว (thr = 1) สาเหตุที่เป็นไปได้คือ glibc เก็บ stack ที่คืนแล้วไว้ใน cache เพื่อใช้ซ้ำกับ thread ถัดไป

ตารางสรุปตามชนิด (กด `t`)

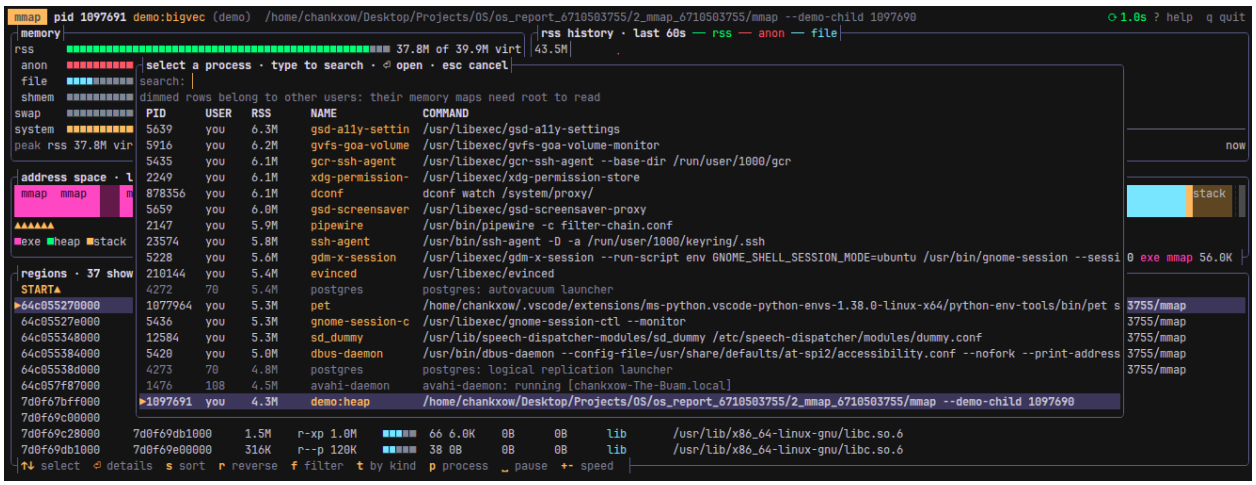


รวมขนาด RSS PSS swap dirty ของแต่ละชนิด ในภาพ `anon` คิดเป็น 94.3% ของ RSS ส่วน `guard` มี virtual 128 MiB แต่ RSS 0 B เพราะเป็น address ที่จองไว้เฉยๆ และ `lib` มี RSS 2.0 MiB แต่ PSS เพียง 66 KiB เพราะหน้าของ libc ถูกแชร์กับโปรเซสอื่นจำนวนมากรายละเอียด region (กด `Enter`)



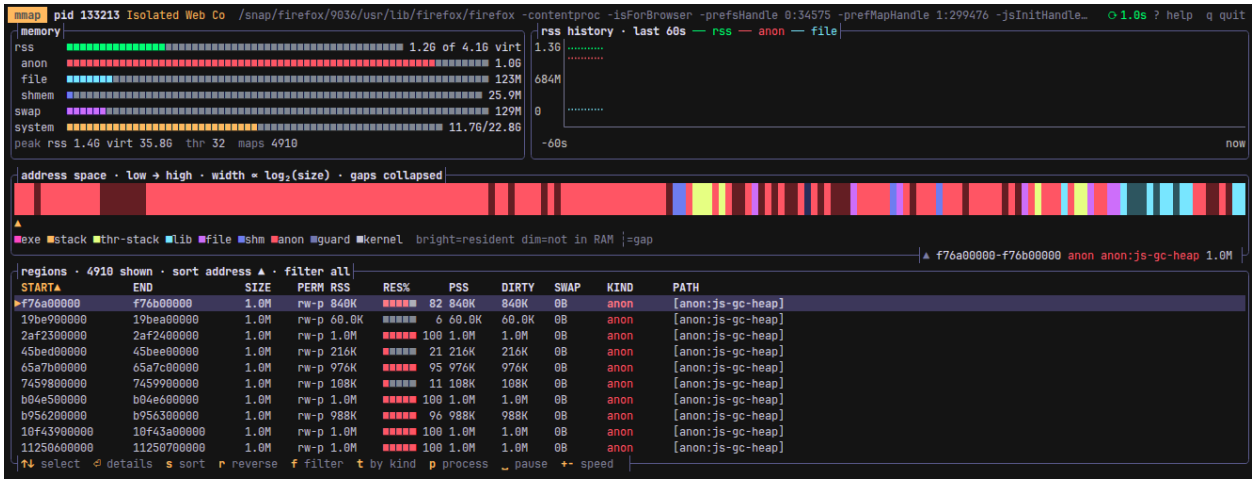
แสดงทุกฟิลด์ของ `smaps` พร้อมคำอธิบายสิทธิ์และชนิด ตัวอย่างในภาพคือ region แรกของตัวโปรแกรม มี Rss 56 kB แต่ Pss 28 kB และ Shared_Clean 56 kB เพราะโปรเซส demo กับ `mmap` ที่เป็นแม่ใช้ไฟล์โปรแกรมเดียวกัน kernel จึงใช้หน้าเดียวกันจาก page cache และ PSS หารสอง นี่คือตัวอย่างจริงของความต่างระหว่าง RSS และ PSS

หน้าเลือกโปรเซส (กด `p` หรือรันโดยไม่ใส่ `-p`)



รายการโปรเซสเรียงตาม RSS พิมพ์เพื่อค้นหาจาก PID ชื่อ หรือคำสั่งได้ (ภาพนี้ค้นด้วยคำว่า `demo`) โปรเซสของผู้ใช้อื่นแสดงเป็นสีจาง เพราะอ่าน memory map ไม่ได้หากไม่ใช่ root

รายงาน snapshot



พิมพ์ 4 ส่วนตามลำดับ: ค่าจาก `status`, สรุปตามชนิดพร้อมบรรทัดตรวจสอบกับ VmRSS, แผนที่แบบตัวอักษร (ตัวใหญ่ = อยู่ใน RAM, ตัวเล็ก = ไม่อยู่, `·` = ช่องว่าง) และตารางทุก region ในภาพผลรวม Rss ทุก region = 90,001,408 B ตรงกับ VmRSS พอดี เหมาะกับการ redirect ลงไฟล์เพื่อเก็บหลักฐานหรือเทียบก่อน-หลัง

โปรเซส demo และแนวคิด OS ที่แต่ละขั้นแสดง

`mmap --demo` รันตัวเองซ้ำเป็นโปรเซสลูก ทำงานเป็นรอบละประมาณ 20 วินาที (ขั้นละ 0.8 วินาที) และตั้งชื่อโปรเซสด้วย `prctl(PR_SET_NAME)` ตามขั้นที่กำลังทำ จึงเห็นชื่อขั้นที่มุมซ้ายบนของหน้าจอ

ขั้น	สิ่งที่ทำ	สิ่งที่เห็นในเครื่องมือ	แนวคิด OS
`demo:heap`	จอง 16 KiB × 200 ก้อน (~3 MiB)	`[heap]` ขยายใหญ่ขึ้น	glibc ใช้ `brk()` กับคำขอ < 128 KiB
`demo:bigvec`	จอง 32 MiB × 2 ก้อน	region anon ใหม่ (อาจถูกรวมเป็น 64 MiB)	glibc ใช้ `mmap()` กับคำขอ ≥ 128 KiB (`M_MMAP_THRESHOLD`), VMA merging
`demo:mmap`	`mmap()` 64 MiB แล้วเขียนทีละ 8 MiB	VmSize เพิ่มทันที RSS ค่อยๆ เพิ่ม	demand paging, page fault
`demo:file`	`mmap()` ไฟล์ระบบแบบอ่านอย่างเดียว	region `file` ใหม่ นับเป็น RssFile	memory-mapped file, page cache
`demo:threads`	สร้าง 2 thread stack ตัวละ 1 MiB	คู่ `guard` + `thr-stack` 2 คู่ thread = 3	thread แชร์ address space แต่มี stack แยก
`demo:stack`	เรียกฟังก์ชันซ้อน 25 ชั้น ชั้นละ 64 KiB	`[stack]` ขยาย ~1.6 MiB และไม่หดกลับ	stack ขยายอัตโนมัติเมื่อเกิด fault ใต้ stack

ขั้น	สิ่งที่ทำ	สิ่งที่เห็นในเครื่องมือ	แนวคิด OS
`demo:free`	คืนทุกอย่างและเรียก `malloc_trim()`	region หายไป RSS ลดลง	`munmap()` คืนหน่วยความจำให้ OS ทันที

2.4 วิธีคอมไพล์และใช้งาน

คำสั่ง `make` คำสั่งเดียวสร้างไฟล์ `./mmap` ที่พร้อมใช้ ต้องมี Rust (`cargo`, `rustc` ≥ 1.75) และใช้อินเทอร์เน็ตเฉพาะครั้งแรกเพื่อดาวน์โหลด crate `ratatui` และ `libc` ตามเวอร์ชันที่ล็อกไว้ใน `Cargo.lock` (ทดสอบกับ Ubuntu 24.04, rustc 1.75.0)

```
sudo apt install build-essential cargo    # ติดตั้ง Rust จาก apt (หรือผ่าน rustup.rs)
cd 2_mmap_6710503755
make            # cargo build --release --locked แล้วคัดลอกมาเป็น ./mmap
make test       # unit test 10 ตัว
make clean      # ลบ target/ และ ./mmap
```

ตัวอย่างการใช้

```
./mmap                          # เปิดหน้าเลือกโปรเซส แล้วเข้าหน้าจอเวลาจริง
./mmap -p 1234                  # ดูโปรเซส PID 1234 แบบเวลาจริง
./mmap --demo                   # รันโปรเซสตัวอย่างแล้วดูหน่วยความจำของมัน
./mmap -p $$ --snapshot         # รายงานข้อความของ shell ปัจจุบัน
./mmap -p $$ -s --no-color > snap.txt   # บันทึกเป็นไฟล์ข้อความล้วน
```

ตัวเลือก	ความหมาย	ค่าเริ่มต้น
`-p PID`	โปรเซสที่ต้องการดู	เปิดหน้าเลือกโปรเซส
`-s`, `--snapshot`	พิมพ์รายงานครั้งเดียวแล้วจบ	ปิด
`--demo`	รันโปรเซสตัวอย่างแล้วดูโปรเซสนั้น	ปิด
`-i MS`	รอบการอัปเดตหน้าจอ (200–10000 ms)	1000
`-w N`	จำนวนช่องของแผนที่ต่อบรรทัดใน snapshot (40–400)	96
`--no-color`	snapshot ไม่ใส่สี (ปิดอัตโนมัติเมื่อ redirect หรือมี `NO_COLOR`)	ปิด

ปุ่มในหน้าจอเวลาจริง

ปุ่ม	การทำงาน
↑ ↓ `j` `k` ล้อเมาส์	เลื่อนแถวที่เลือก

ปุ่ม	การทำงาน
`Enter` หรือคลิกแถวเดิมซ้ำ	ดูทุกฟิลด์ smaps ของ region นั้น
คลิกบนแผนที่	เลือก region ที่ตำแหน่งนั้น
`s` / `r`	เปลี่ยนคอลัมน์ที่เรียง (address, size, RSS, PSS, dirty, swap) / กลับลำดับ
`f`	กรองตามชนิด region
`t` / `Tab`	สลับตาราง region กับตารางสรุปตามชนิด
`p`	เลือกโปรเซสอื่น (พิมพ์ค้นหาจาก PID ชื่อ หรือคำสั่ง)
`space` / `u`	หยุด-เล่นต่อ / อัปเดตทันที
`+` / `-`	อัปเดตช้าลง / เร็วขึ้น
`?`	หน้าช่วยเหลือ
`q` `Esc` `Ctrl+C`	ออกจากโปรแกรม

ค่าตัวเลือกที่ผิดรูปแบบ อยู่นอกช่วง หรือ PID ที่ไม่มีอยู่ จะถูกปฏิเสธพร้อมข้อความ error และ exit code 1 เช่น `./mmap -p 999999 --snapshot` พิมพ์ `mmap: no process with PID 999999`

2.5 การทดสอบ ข้อจำกัด และความปลอดภัยต่อระบบ

ทุกการทดสอบผ่าน และค่าที่เครื่องมือแสดงตรงกับตัวนับของ kernel เอง (ผลรวม Rss ทุก region = VmRSS พอดีในการทดสอบกับ shell)

การทดสอบ	ผล
`make` จากสำเนาสะอาด ด้วย rustc 1.75.0	คอมไพล์ผ่านใน 14.5 วินาที ได้ `./mmap`
`make test` (unit test 10 ตัว: parse maps/smaps/status, จัดชนิด region, วางแผนที่, จัดรูปแบบขนาด)	ผ่าน 10/10
snapshot ของ shell ตัวเอง	sum of per-region Rss = 4,554,752 B = VmRSS
`-p 999999` (PID ที่ไม่มี)	`no process with PID 999999`, exit code 1
`-p 1` (systemd ของ root)	แสดงค่าจาก status พร้อมคำเตือน permission denied ไม่ crash
`--demo --snapshot`	รายงานโปรเซส demo ขณะอยู่ที่ขั้น `demo:mmap` (RSS 77.9 MiB) แล้ว demo ถูก kill
เปิดหน้าจอเวลาจริงแล้วกด `q`	เทอร์มินัลกลับสภาพเดิม ไม่มีโปรเซส demo ค้าง

ข้อจำกัด

- **ดูได้เฉพาะโปรเซสของผู้ใช้เอง** kernel ตรวจสิทธิ์การอ่าน `maps`/`smaps` แบบเดียวกับ ptrace โปรแกรมจึงตั้งใจไม่รองรับการรันด้วย `sudo`
- **ค่าไม่ได้อ่านพร้อมกันทุกไฟล์** `status` กับ `smaps` อ่านคนละจังหวะ และ VmRSS เป็นตัวนับที่ kernel อัปเดตแบบแบ่งก้อนต่อ CPU ผลรวมจึงอาจต่างกันเล็กน้อยกับโปรเซสที่กำลังเปลี่ยนแปลง
- **การอ่าน smaps มีต้นทุน** kernel ต้องเดิน page table ของทุก region ทุกครั้งที่อ่าน โปรเซสที่มี region นับพัน (เช่นเบราว์เซอร์) ควรเพิ่มรอบอัปเดตด้วย `-i 2000`
- **การจัดชนิดบางอย่างเป็นการอนุมาน** `thr-stack` อาศัยตำแหน่งที่ติดกับ guard page และ region `guard` ขนาดเกือบ 64 MiB คือพื้นที่ที่ glibc จองไว้ให้ malloc arena ของ thread ไม่ใช่ guard page จริง
- **VMA merging** kernel รวม region anonymous ที่อยู่ติดกันและสิทธิ์เหมือนกันเป็น region เดียว เช่น Vec 32 MiB สองก้อนใน demo อาจแสดงเป็น 64 MiB ก้อนเดียว

ความปลอดภัยต่อระบบ (DO NO HARM)

- อ่านไฟล์ใน `/proc` อย่างเดียว ไม่ใช้ `ptrace` ไม่อ่านหรือเขียนหน่วยความจำของโปรเซสอื่น ไม่ต้องใช้ root และไม่เขียนไฟล์ลงดิสก์
- คืนสภาพเทอร์มินัล (ออกจาก alternate screen, ปิด raw mode และ mouse capture) ทุกครั้งที่ออก รวมกรณี panic ผ่าน panic hook
- event loop รอ input ด้วย `poll` แบบมี timeout ไม่วน busy loop
- โปรเซส demo ใช้หน่วยความจำไม่เกินประมาณ 140 MiB และมีกลไกหยุด 4 ชั้น: `DemoGuard` kill ลูกเมื่อ `mmap` จบ (รวม panic), `PR_SET_PDEATHSIG` ให้ kernel ส่ง SIGTERM เมื่อแม่ตาย, ตรวจ parent PID ทุก 0.8 วินาที และหยุดเองหลัง 30 นาที
- ตรวจค่าอาร์กิวเมนต์ทุกตัว (เช่น interval 200–10000 ms กันการอ่าน `/proc` ถี่เกินไป)

การรันบน Windows และ macOS

ทั้งสองข้อพัฒนาและวัดผลบน Linux หลังจากนั้นได้เพิ่มวิธีรันบน Windows และ macOS โดยแก้เฉพาะ `Makefile`, `BUILD.md` และ `README.md` ไม่ได้แก้ซอร์สโค้ด ผลการทดลองทั้งหมดในรายงานจึงยังเป็นของ Linux เหมือนเดิม

	Linux	Windows 10/11	macOS
ข้อ 1 `parallel` (C + OpenMP)	`make`	รันใน WSL2 ด้วย `make`	`make` (Apple clang + Homebrew `libomp`) ค่าหน่วยความจำบางค่าไม่ถูกต้อง
ข้อ 2 `mmap` (Rust)	`make`	รันใน WSL2 ด้วย `make`	`make docker` (รันใน Linux container)

เหตุผลที่รันบน Windows/macOS โดยตรงไม่ได้ทั้งหมด

สิ่งที่โปรแกรมใช้	Linux	macOS	Windows	ผลกระทบ
OpenMP runtime	libgomp มากับ gcc	Apple clang ไม่มี ต้องติดตั้ง `libomp`	มีใน MinGW/MSVC	macOS ต้องใช้ flag `-Xpreprocessor -fopenmp -lomp`
`getrusage()` จาก `<sys/resource.h>` (ข้อ 1)	มี `ru_maxrss` หน่วย KiB	มี แต่ `ru_maxrss` หน่วย byte	ไม่มี	macOS: Peak RSS สูงเกินจริง 1024 เท่า / Windows: คอมไพล์ไม่ผ่าน
`/proc/self/status` (ข้อ 1, VmPeak)	มี	ไม่มี	ไม่มี	VmPeak ใช้ได้เฉพาะ Linux
`/proc/PID/maps`, `smaps` (ข้อ 2)	มี	ไม่มี (ใช้ Mach API เช่น `mach_vm_region`)	ไม่มี (ใช้ `VirtualQueryEx`)	ข้อ 2 ต้องเขียนส่วนอ่านข้อมูลใหม่ทั้งหมดจึงจะรันได้โดยตรง

ข้อ 2 อ่าน memory map ผ่าน `/proc` ซึ่งเป็นอินเทอร์เฟซเฉพาะของ Linux kernel macOS และ Windows เก็บข้อมูลเดียวกันไว้ แต่เปิดให้อ่านผ่าน system call คนละแบบ และบน macOS การอ่านหน่วยความจำของโปรเซสอื่นต้องใช้ `task_for_pid` ซึ่งถูกจำกัดสิทธิ์เข้มกว่า จึงเลือกรันข้อ 2 บน Linux ต่อไป (WSL2 หรือ container) แทนการ port โค้ด

สิ่งที่แก้ไขและเพิ่มเข้ามา

ไฟล์	สิ่งที่เปลี่ยน
`1_parallel/Makefile`	ตรวจระบบด้วย `$(OS)` และ `uname -s` — Linux: `gcc -fopenmp` เหมือนเดิม / macOS: `clang -Xpreprocessor -fopenmp` + `libomp` จาก `brew --prefix libomp` (หรือ `make CC=gcc-14` ใช้ gcc ของ Homebrew) / Windows: หยุดพร้อมแจ้งให้ใช้ WSL2

`2_mmap/Makefile`	`make` และ `make test` หยุดพร้อมบอกทางเลือกเมื่อไม่ใช่ Linux และเพิ่ม target ใหม่ `make docker`

ผลการทดสอบ

ทดสอบบนเครื่อง Linux และใน container เมื่อ 13 ต.ค. 2026 ยังไม่ได้ทดสอบบนเครื่อง Mac และ Windows จริง

การทดสอบ	วิธี	ผล
ข้อ 1 บน Ubuntu 24.04 ที่ติดตั้งใหม่ (แทน WSL2)	`make` แล้ว `-v` ทั้ง 4 อัลกอริทึม	ผ่าน
ข้อ 1 ทางที่ macOS ใช้	clang + libomp ใน container, `-v` ทั้ง 4 อัลกอริทึม	ผ่าน
ข้อ 1 macOS กับ gcc ของ Homebrew	`make CC=gcc` (จำลอง)	รอบแรกพบ bug คือยังบังคับให้มี libomp แก้แล้วผ่าน
ข้อ 1 macOS ที่ไม่มี libomp / Windows	จำลองค่า `UNAME_S` และ `OS`	หยุดพร้อมบอกวิธีแก้ถูกต้อง
ข้อ 2 บน Ubuntu 24.04 ที่ติดตั้งใหม่ (แทน WSL2)	`make`, `make test`, `--demo --snapshot`	ผ่าน 10/10, Rss รวมทุก region = VmRSS (90,193,920 B)
ข้อ 2 `make docker` แบบ snapshot	รันจริง	ผ่าน ใช้เวลา 42 วินาทีครั้งแรก, Rss รวม = VmRSS (89,878,528 B), โฟลเดอร์งานไม่เปลี่ยน
ข้อ 2 `make docker` แบบ live (TUI)	รันใน tmux แล้วกด `q`	แสดงผลถูกต้อง, exit 0, ไม่มี container ค้าง
ข้อ 2 บนระบบที่ไม่ใช่ Linux	จำลองค่า `UNAME_S`	หยุดพร้อมบอกทางเลือก WSL2 / `make docker`

ข้อจำกัดที่เหลือ: บน macOS ข้อ 1 คำนวณและจับเวลาถูกต้อง แต่ Peak RSS สูงเกินจริง 1024 เท่าและ VmPeak ใช้ไม่ได้ แก้ได้ด้วยการแก้ `parallel.c` ประมาณ 3 บรรทัด ซึ่งยังไม่ได้ทำ นอกจากนี้เวลาที่วัดบน macOS, WSL2 หรือ VM จึงไม่ได้นำไปเทียบกับผลในรายงานโดยตรง

การใช้ งาน AI

ทั้งสองข้อใช้ Claude Code (Anthropic) ช่วยเขียนโค้ด โดยไม่ได้ใช้ไฟล์ SKILL จึงไม่มีไฟล์ SKILL ในโฟลเดอร์ส่งงาน

งาน	ส่วนที่ AI เขียน	ส่วนนิสิต
ข้อ 1	`parallel.c`, `bench.sh`, `plot.py`, `Makefile`, `BUILD.md`	เกำหนดการทดลองที่ต้องการ, รัน benchmark บนเครื่องตัวเอง, ตรวจโค้ดและผลลัพธ์ กับการออกแบบวางระบบ ไอเดีย แบบ sanwich
ข้อ 2	ไฟล์ `.rs` ทั้งหมด, `Cargo.toml`, `Makefile` (`Cargo.lock` สร้างโดย cargo)	กำหนดความสามารถและหน้าตาโปรแกรม, ทดสอบกับโปรเซสจริง ออกแบบไอเดีย วางระบบที่จำเป็น
รายงาน	ใช้ตรวจหาคำผิดและช่วยสรุปผลการทดลองกละช่วยสร้างการ plot กราฟจากข้อที่ 1 แคปภาพช่วย plan review promt ไอเดียให้ถูกต้องนการ นำมาช่วยพัฒนาระบบ	เนื้อหาและการวิเคราะห์ทั้งหมด

การตรวจสอบผลงานของ AI

- ข้อ 1: รัน `-v` เทียบกับการคำนวณ serial ทุกอัลกอริทึม และตรวจว่า checksum ตรงกันทุกการรันในข้อมูล benchmark (หัวข้อ 1.2)
- ข้อ 2: รัน unit test และตรวจว่าผลรวม Rss ตรงกับ VmRSS ของ kernel (หัวข้อ 2.5)
- ตรวจเงื่อนไข DO NO HARM ด้วยตนเอง: ไม่ต้องใช้ root, จำกัดหน่วยความจำ และไม่มีโปรเซสค้างหลังปิดโปรแกรม

เอกสารอ้างอิง

ข้อ 1

1. Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the AFIPS Spring Joint Computer Conference*,483–485.doi:10.1145/1465482.1465560

2. Gustafson, J. L. (1988). Reevaluating Amdahl's law. *Communications of the ACM*, 31(5), 532–533. doi:10.1145/42411.42415
3. Karp, A. H., & Flatt, H. P. (1990). Measuring parallel processor performance. *Communications of the ACM*, 33(5), 539–543. doi:10.1145/78607.78614
4. Lam, M. S., Rothberg, E. E., & Wolf, M. E. (1991). The cache performance and optimizations of blocked algorithms. *ASPLOS IV*, 63–74. doi:10.1145/106972.106981
5. OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*. openmp.org/specifications
6. GNU Project. *GNU libgomp manual* (Environment variables: OMP_PROC_BIND, OMP_PLACES, OMP_WAIT_POLICY, OMP_STACKSIZE). gcc.gnu.org/onlinedocs/libgomp
7. Drepper, U. (2007). *What Every Programmer Should Know About Memory*. Red Hat. akkadia.org/drepper/cpumemory.pdf
8. Linux kernel source, `kernel/sched/core.c` (`sched_prio_to_weight`). github.com/torvalds/linux

ข้อ 2

9. Linux man-pages project. *proc_pid_maps(5)*, *proc_pid_smaps(5)*, *proc_pid_status(5)*. man7.org/linux/man-pages
10. The Linux Kernel documentation. *The /proc Filesystem*. docs.kernel.org/filesystems/proc.html
11. Linux man-pages project. *mallopt(3)* (M_MMAP_THRESHOLD), *mmap(2)*, *prctl(2)* (PR_SET_NAME, PR_SET_PDEATHSIG). man7.org/linux/man-pages
12. Gorman, M. (2004). *Understanding the Linux Virtual Memory Manager*. Prentice Hall. kernel.org/doc/gorman
13. Ratatui contributors. *Ratatui documentation*. ratatui.rs